

The background features a complex geometric design. On the left, there are light gray hexagonal outlines. Overlaid on these and extending to the right are semi-transparent blue hexagonal outlines. Within these hexagons, there are small green and blue triangles pointing in various directions. On the right side of the image, there is a close-up photograph of a green printed circuit board (PCB) with various electronic components, including a large integrated circuit and several connectors. The overall color palette is dominated by light grays, blues, and greens.

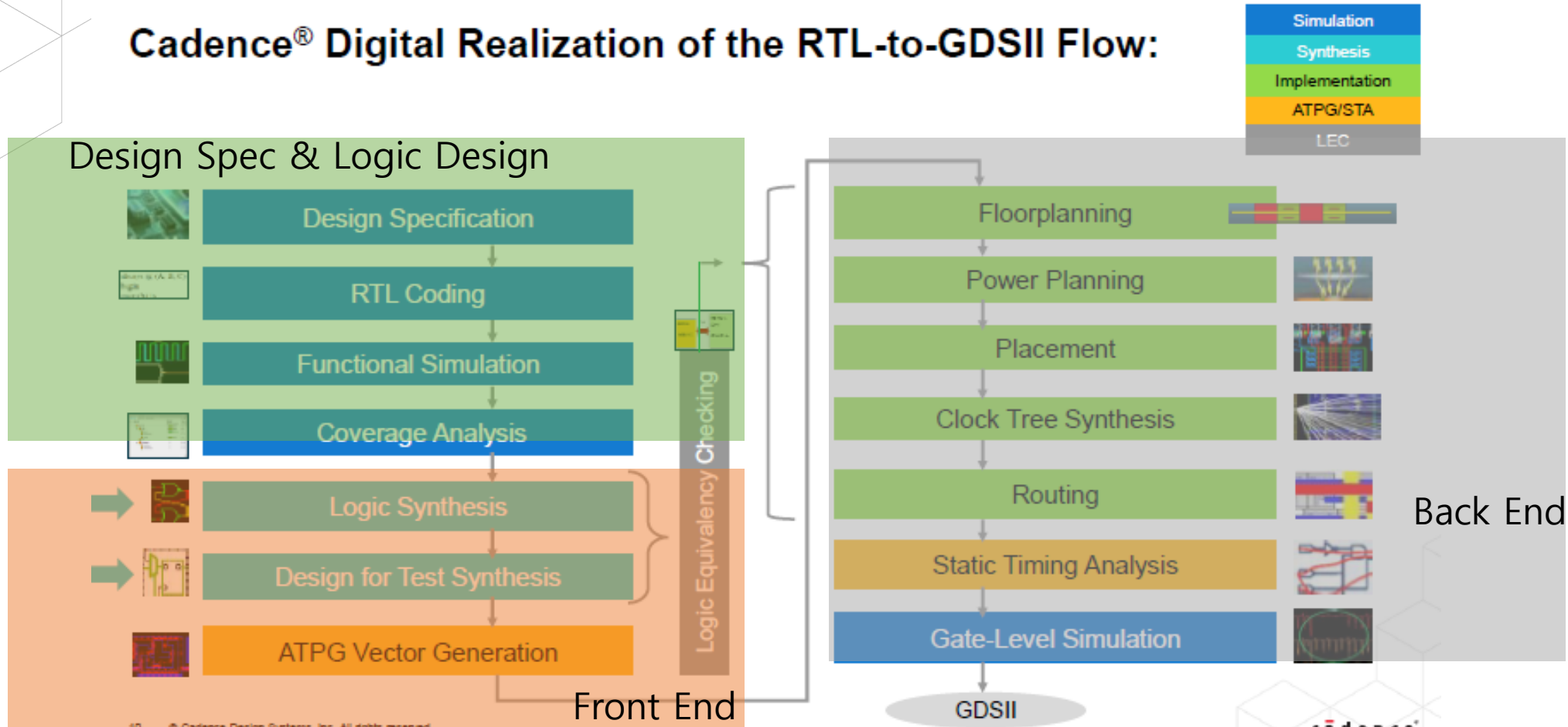
Digital Implementation 101

Session1 : Synthesis

1. RTL-to-GDSII Implementation Flow

Overview: Digital Realization of the RTL-to-GDSII Flow

Cadence® Digital Realization of the RTL-to-GDSII Flow:



2. Overview

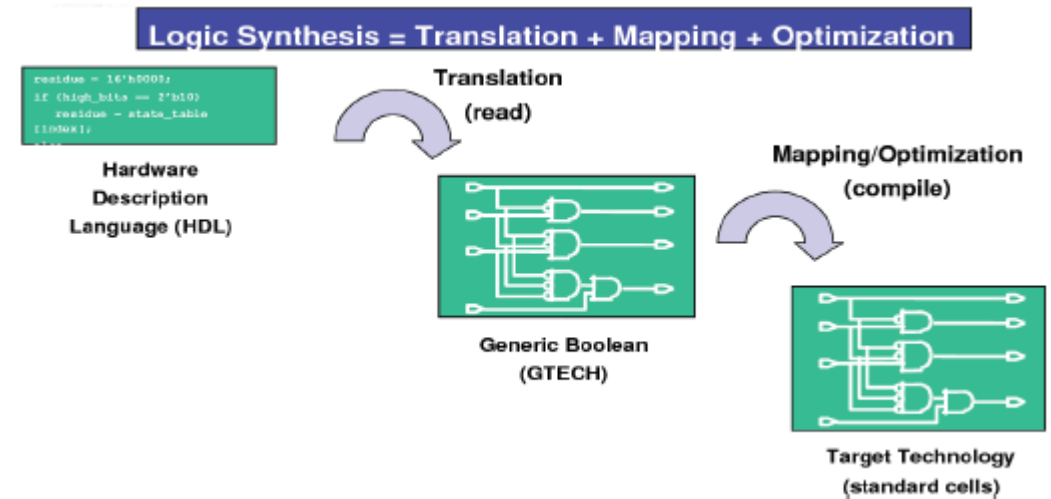
Notion of Synthesis

- **Logic Design**

- 회로의 기능적 디자인을 묘사함
- RT Level: logic/arithmetic operation, control flow...

- **Synthesis**

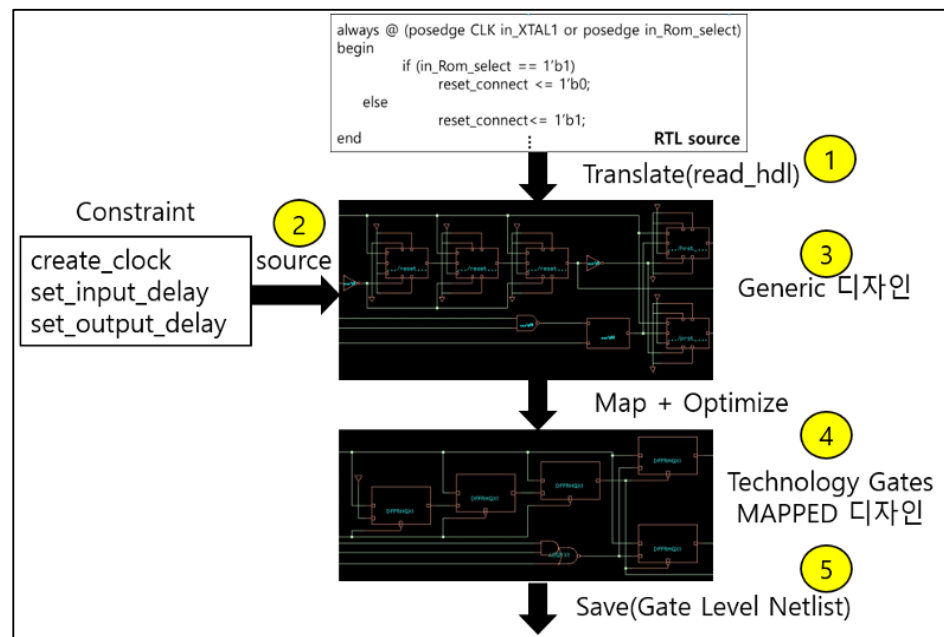
- RT Level의 HDL의 코드를 Gate Level의 Netlist로 변환
- 변환시에 technology library 및 optimization constraints를 고려



Necessity of Synthesis

- 합성이 필요한 이유

- 구현 가능한 형태로 변환
 - 동작을 기술하는 RTL 코드를 실제 하드웨어인 물리적 게이트로 변환
- 최적화
 - 면적, 타이밍, 전력을 줄이기 위한 최적화 수행
- 제약 조건 만족
 - 클록 주기, 면적 제한 등 설계 요구사항을 만족하는 회로 생성



Specific Steps of Synthesize(1)

- **1단계 : Translation(번역)**

- RTL 코드를 툴이 인식할 수 있도록 변환
- 계층적 구조(Hierarchical Structure)를 파악 및 인식
- Generic Boolean Gates 형태로 초기 변환

- **2단계 : Constraint 적용(제약 조건 설정)**

- 설계 목표인 면적, 타이밍, 환경 조건 등을 구체적으로 정의
- 합성 툴이 이 제약 조건을 기준으로 최적화 수행
- 합성의 방향성을 명확히 설정하는 핵심 단계

Specific Steps of Synthesize(2)

- **3단계 : Optimization & Mapping(최적화 및 매핑)**
 - 기존 디지털 로직을 효율적인 구조로 재구성
 - 공정사 제공 Standard Cell 라이브러리와 매핑
 - 출력 강도 조정 (4x, 3x, 2x, 8x 등) 및 셀 간 연결 최적화
- **4단계 : Save(저장)**
 - 최적화된 설계를 게이트 레벨 넷리스트(Gate-Level Netlist)로 저장
 - 다음 단계 툴에서 사용할 수 있는 형태로 결과물 생성

Transformation of RTL Code : 3 Steps



Synthesis is the process of transforming your RTL design (high-level description) into a gate-level netlist, given all the specified constraints and optimization settings.

Generic Optimization

```
syn_generic <-  
physical>
```

Generic – Generic gate optimization.

```
wire n_66, n_67, n_68, n_69, n_70;  
COM_flop \count_reg[0] (.clk (clk), .d (n_11), .ena (1'b1), .aclr  
    (n_53), .apre (1'b0), .srl (1'b0), .srd (1'b0), .q (count[0]));
```

A generic gate is used for the instance Count_reg[0].

Technology
Transformation
(Mapping)

```
syn_map <-  
physical>
```

Maps the specified design(s) to the cells described in the *supplied* technology library and performs logic optimization.

```
***** your_synth \syn \count[0] (.d (n_67), .t (n_68));  
DFFRHQX1 \count_reg[0] (.RN (rst), .CK (clk), .D (n_66), .Q  
    (count[0]));
```

Maps the generic gate with std cell DFFRHQX1 for the instance Count_reg[0].

Incremental
Optimization

```
syn_opt <-  
spatial> <-incr>
```

Optimization – Synthesizes the design to optimized gates.

```
*****  
NOR2BX1 g203_8246 (.AN (count[1]), .B (n_68), .Y (n_2));  
DFFRX1 \count_reg[0] (.RN (rst), .CK (clk), .D (n_66), .Q (count[0]),  
    .QN (n_69));
```

Optimize the cell DFFRHQX1 to DFFRX1 for the instance Count_reg[0].

Option *-physical* and *-spatial*

- Requires a Genus-Physical License option.
- Takes physical domain into consideration.
- Runs placement and performs physical optimization.

Transformation of RTL Code : 3 Steps

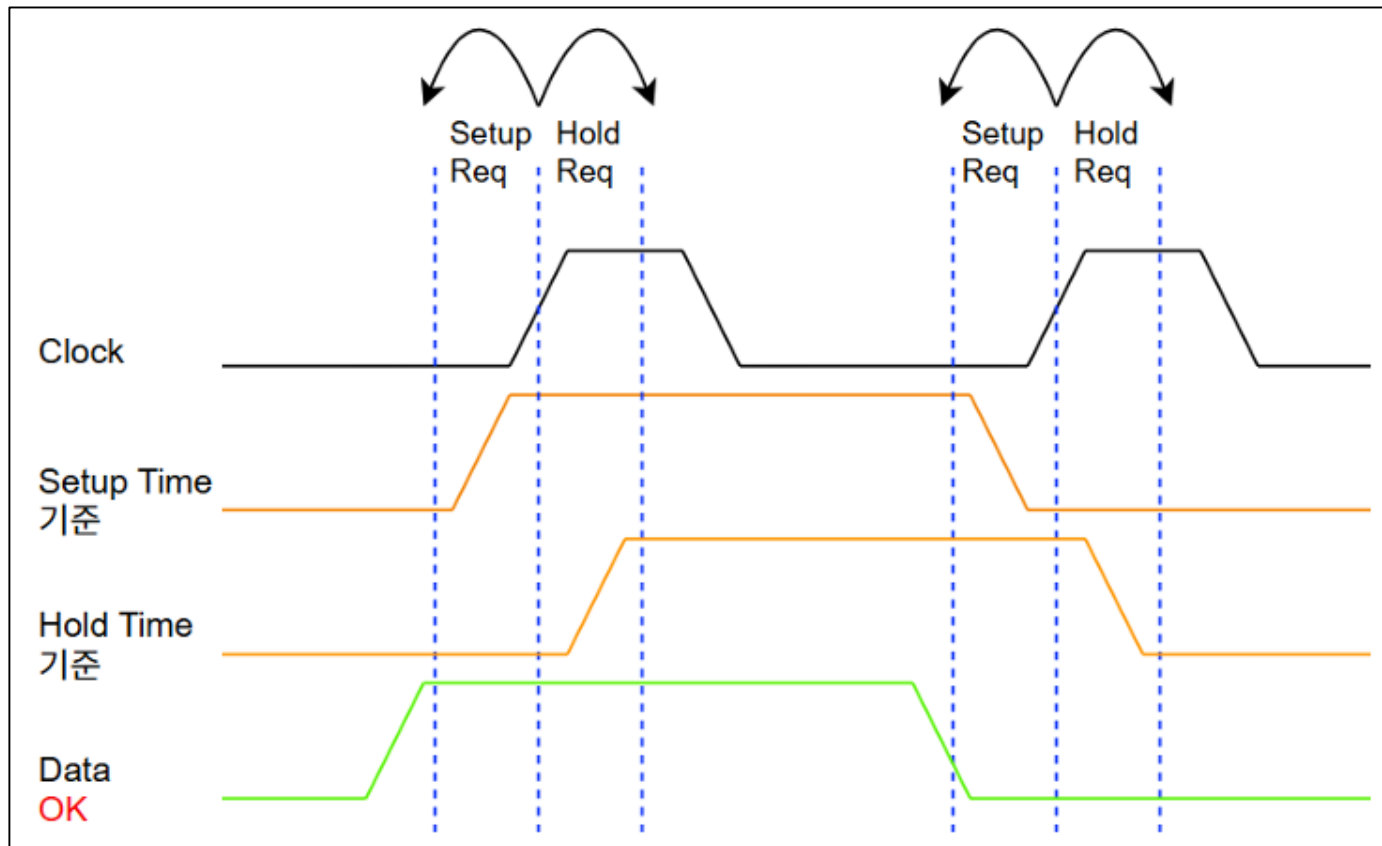
- 합성과정에서 RTL 코드는 세 번의 주요 변환을 거친다.
- **syn_generic(제너릭 합성)**
 - 공정사 라이브러리를 사용하지 않는 기본 논리 최적화
 - RTL 의 인스턴스 이름을 그대로 유지 (예: Count_reg[0])
 - 논리적 구조 최적화에 집중
- **syn_map(매핑 합성)**
 - 공정사 Standard Cell 라이브러리와 매핑
 - 인스턴스 이름이 셀 이름으로 변경 (예: Count_reg[0] → DFFRHQX1)
 - 실제 제조 가능한 게이트로 변환
- **syn_opt(최적화 합성)**
 - 매핑된 디자인의 추가 최적화
 - 타이밍, 면적, 전력 관점에서 최종 조정 (예: DFFRHQX1 → DFFRX1)
 - 제약 조건을 만족하는 최적 솔루션 도출

3. Timing

Setup/Hold Time

- **Setup Time과 Hold Time**

- Setup Time: 클럭 상승 에지 이전에 데이터가 안정적으로 도착해야 하는 시간
- Hold Time: 클럭 상승 에지 이후에도 데이터가 유지되어야 하는 시간



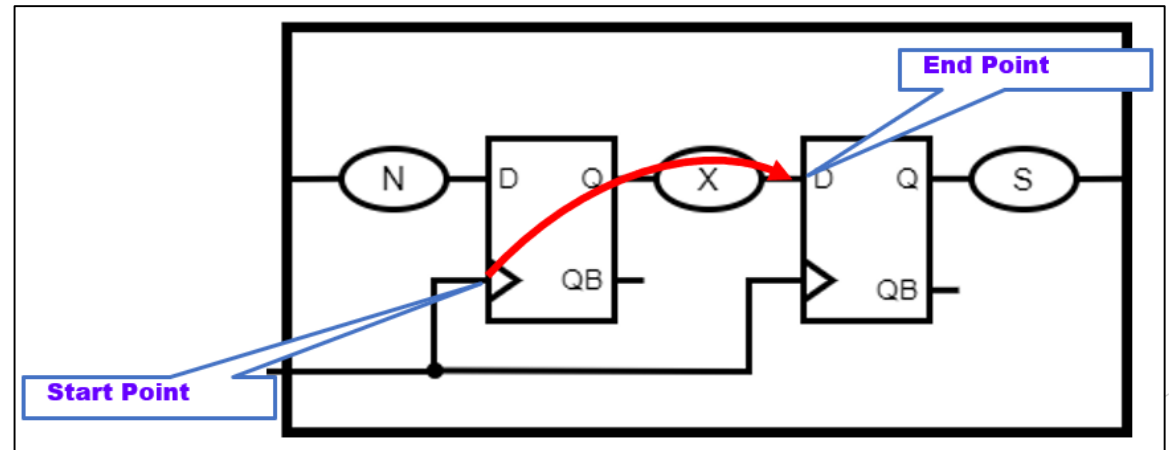
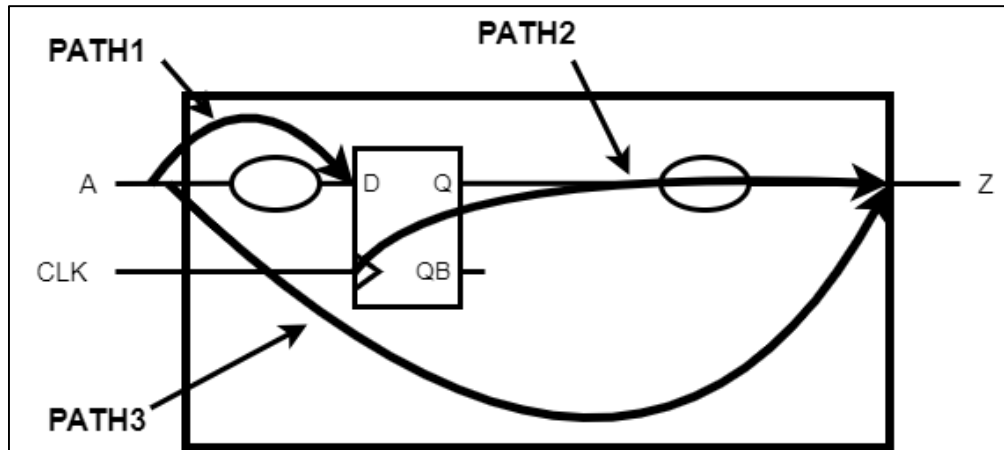
Path

- 디지털 로직은 Verilog 언어로 기술 가능, 언어를 사용한다는 것은 계산 로직의 생성이 가능
- 계산 로직은 신호의 연결이 복잡하므로 많은 EDA 툴에 공통적으로 적용할 기준이 필요
- Path(경로)의 개념
 - 시작점인 Start Point와 종착점인 End Point가 있음
 - Start Point : 입력 포트 또는 레지스터의 클록 핀
 - End Point : 출력 포트 또는 다음 레지스터의 데이터 입력 핀
 - 사람의 손가락 마디처럼, 길게 이어져 있는 신호의 길을 플립플롭 단위로 끊고 이해 및 분석할 수 있도록 하는 중요한 단위

Path

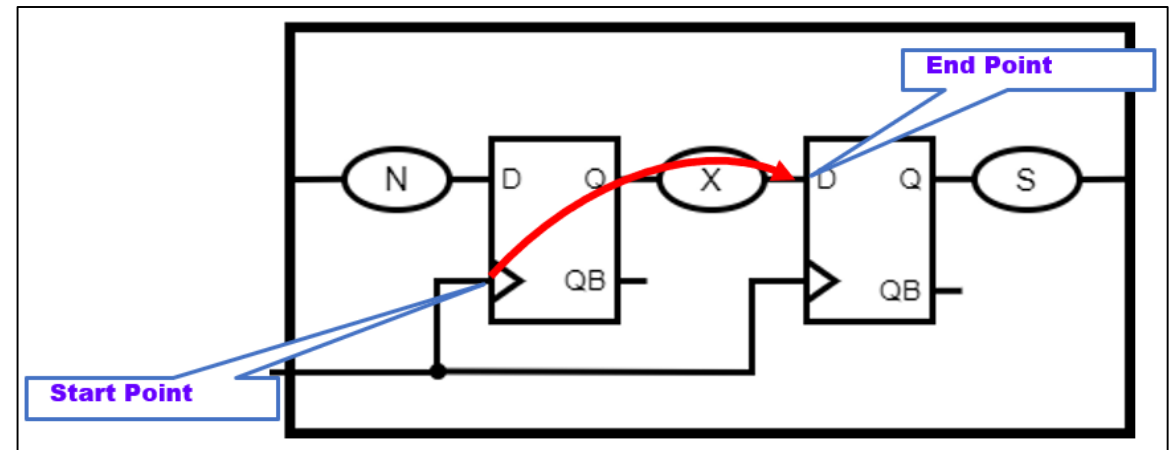
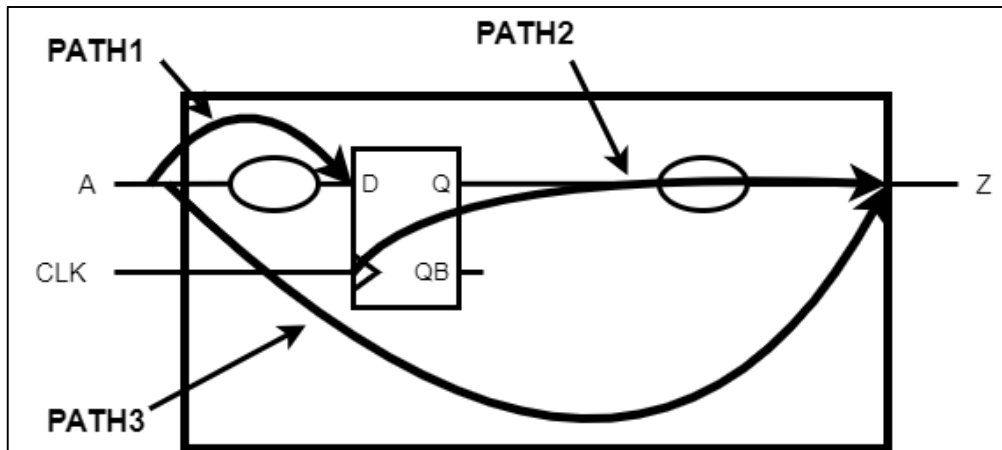
- Path의 예시

- 왼쪽 그림은 툴이 디자인을 읽어 들이면, 모든 Path에 대해서 빠짐없이 분석할 수 있다는 것
- 오른쪽 그림은 툴이 Register 간의 연결에서 Path를 인지하는 원리
 - Start Point는 앞선 플립플롭의 클록 핀
 - End Point는 다음 플립플롭의 D 핀



Path

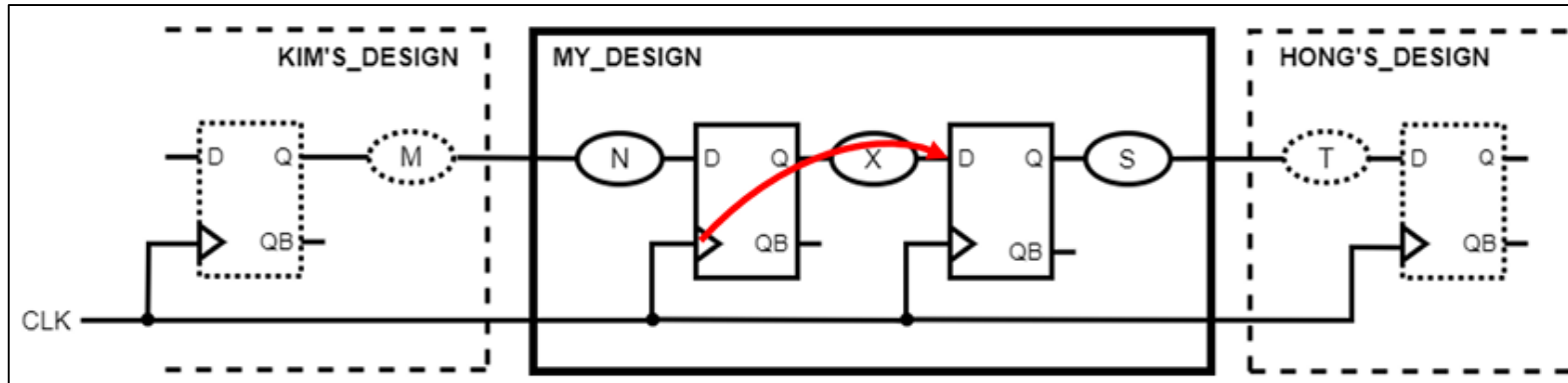
- 디지털 디자인이 아래와 같은 세 종류의 Path로 구분됨
 - Input Path: 외부 입력에서 내부 레지스터로의 경로
 - Internal Path: 레지스터 간 경로 (Reg-to-Reg)
 - Output Path: 내부 레지스터에서 외부 출력으로의 경로



Path

- **MY_DESIGN 의 예시**

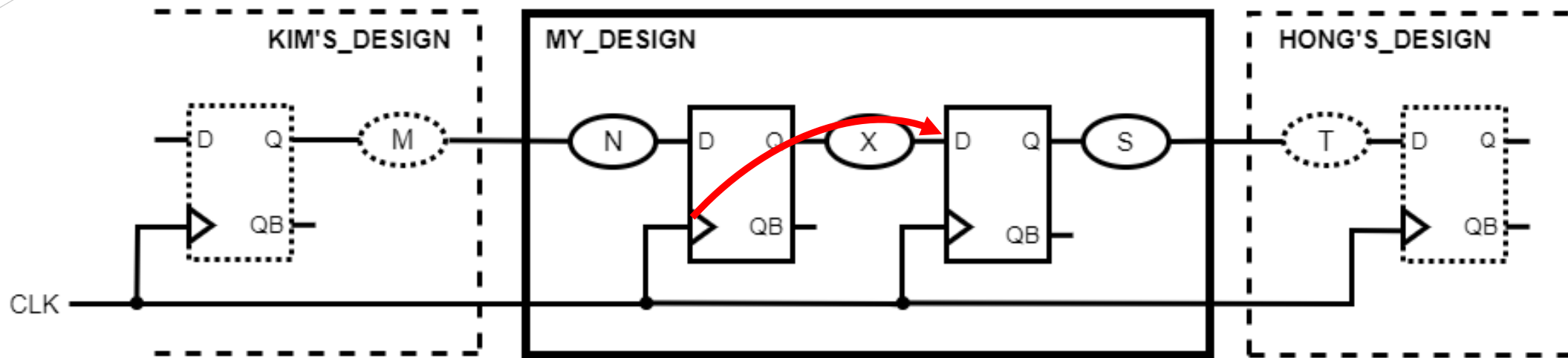
- Input Path : 입력포트부터 N으로 표시된 조합회로를 거쳐 플립플롭의 D까지
- Internal Path : 빨간색 화살표처럼, 첫째 플립플롭의 클록 핀부터 두 번째 플립플롭의 D까지
- Output Path : 두 번째 플립플롭의 클록 핀부터 S로 표시된 조합회로를 거쳐 출력포트까지



- **위 그림처럼 연결되면, 서브모듈이었을 때와 다르게 MY_DESIGN의 Path에 변화발생**

- Input Path : KIM 디자인의 플립플롭 클록 핀부터 M, N 조합회로를 거쳐 첫 번째 플립플롭의 D까지
- Output Path : 두 번째 플립플롭의 클록 핀부터 S, T 조합회로를 거쳐 HONG 디자인 플립플롭의 D까지

Path



시간적 의미 추가

위 그림에서 빨간색 화살표로 정의하고 있는 Internal Path(Reg-to-Reg)는 시간적인 의미도 담고 있다.

Start Point부터 End Point까지 얼마의 시간이 걸렸는지에 대한 것인데, 기본적으로 Cell Delay와 Net Delay의 합으로 정의한다.

Timing

타이밍 = Cell Delay + Net Delay

- Cell Delay : 셀에서 걸린 시간

플립플롭이 클록의 Rising Edge를 인지한 시점부터 출력이 Q핀으로 나오는데 까지 걸리는 CLK-to-Q 시간과 AND, OR 게이트 같은 조합회로의 동작 시간을 포함한다.

- Net Delay : 선(Wire)에서 걸린 시간

셀과 셀은 Net(Wire)로 연결되는데, 디지털 로직처럼 Verilog로 회로를 기술하면 자연스럽게 Net 연결이 방대해질 것이므로 무시할 수 없는 값이 될 것이다.

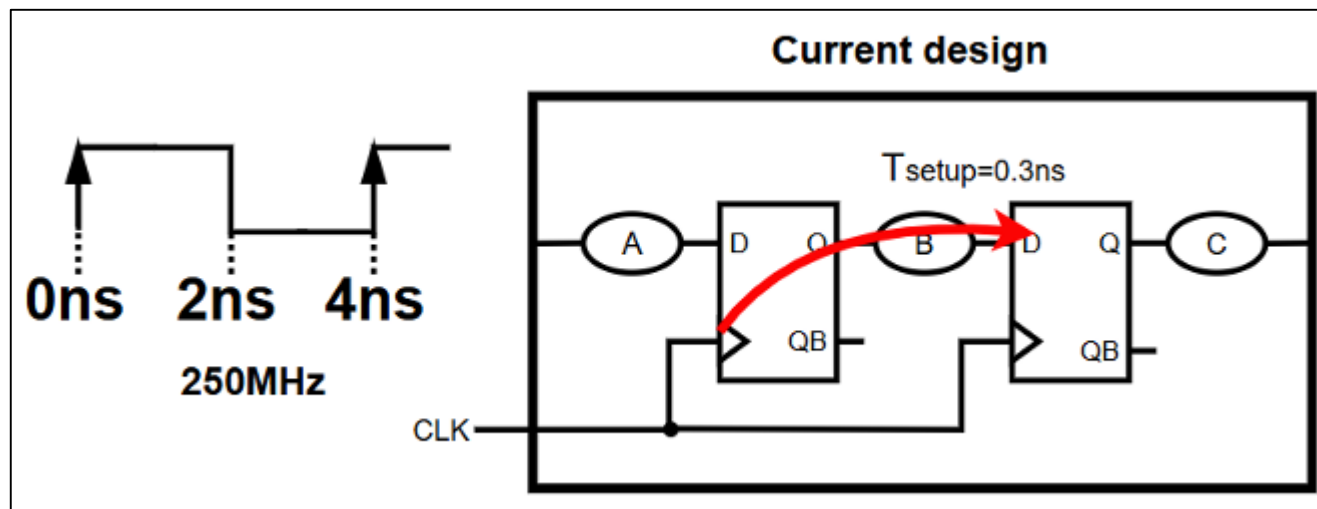
Timing

- Path의 종합적 이해

- 모든 Path는 플립플롭을 기준으로 구분, Path의 시간적 거리는 클록 펄스 한 주기

- 아래 그림의 예

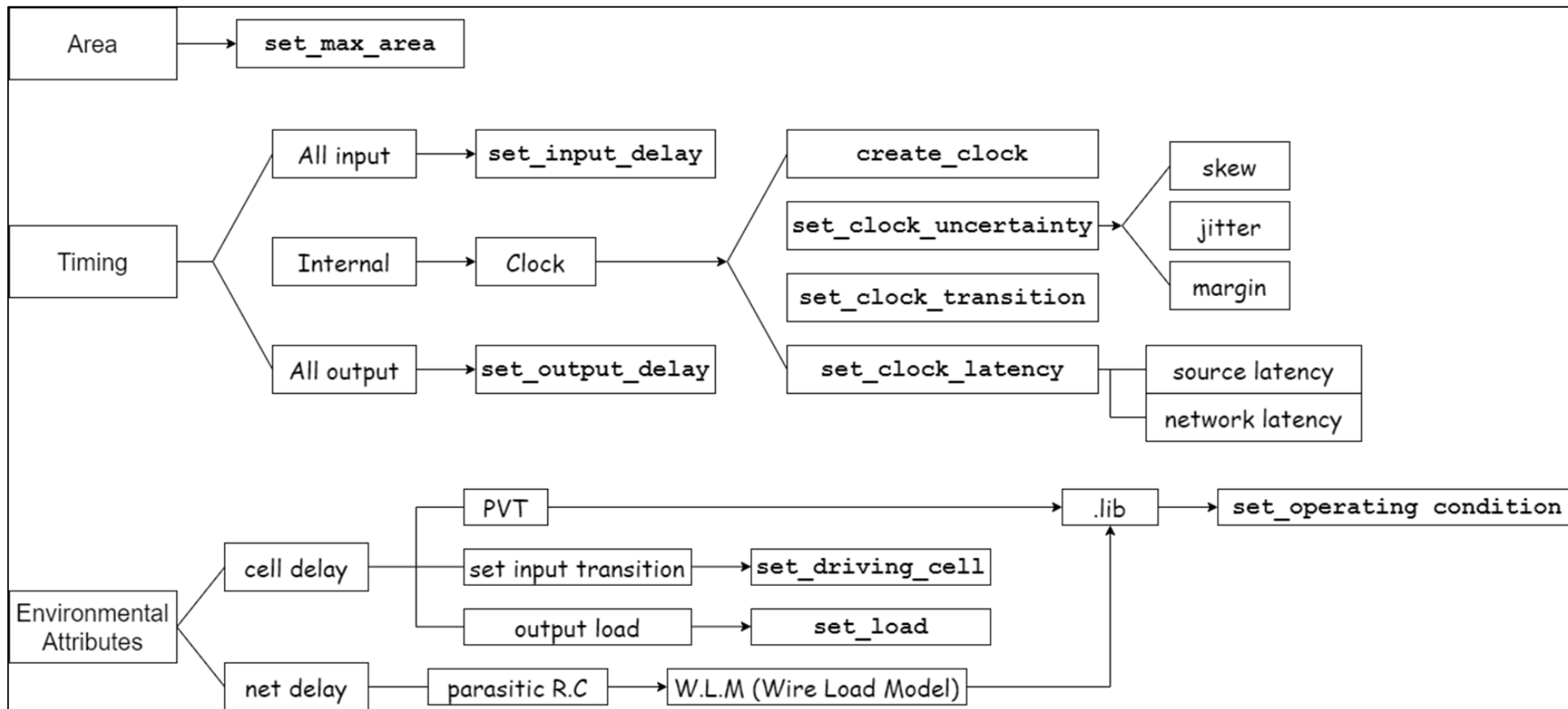
- 주기가 4ns라고 하면
- 앞선 플립플롭 클록 핀부터 다음 플립플롭의 D까지로 정의되는 Path는 4ns 이내에 신호가 도착해야 함
- Path 내의 모든 Cell Delay와 Net Delay를 모두 합한 시간이 4ns보다 적어야만 한다는 것이다.
- Setup Time Requirement가 0.3ns라면 3.7ns가 되기 전에 0 또는 1값이 다음 FF의 D에 도착해야 함



4. Constraints

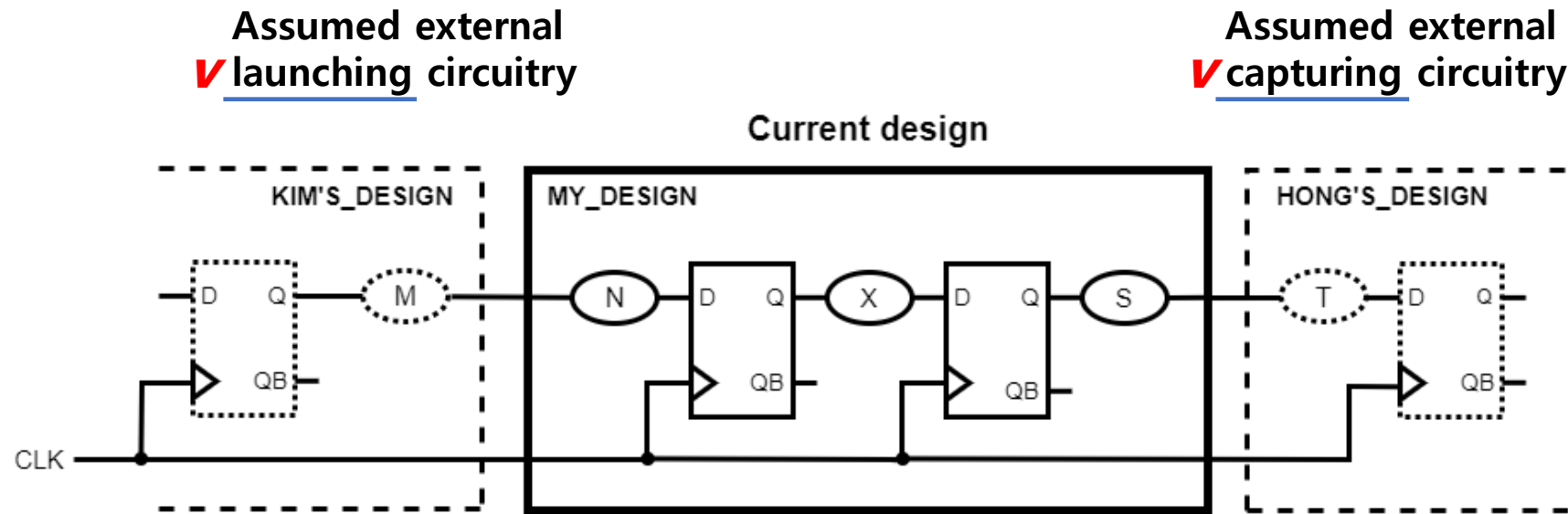
Notion of Constraints

- **Constraint = 제약 조건 = 스펙**
- 칩이 동작할 때 만족해야 하는 사항들을 모두 적어서 EDA 툴에게 알려주면, 툴은 해당 내용들을 모두 만족하는 칩이 될 수 있도록 구현해 줌

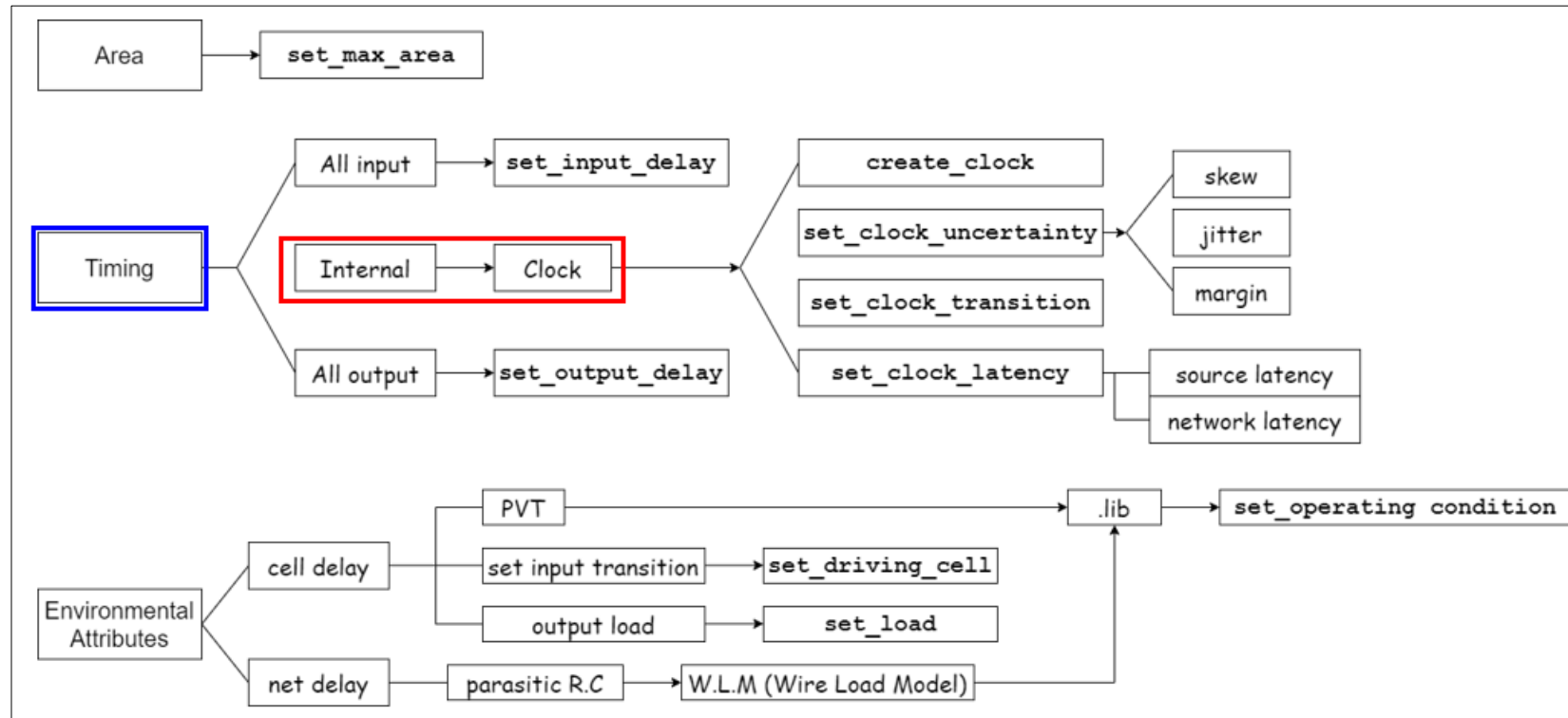


Basic Condition

- 설계는 기본적으로 Single 클록(한 주기 클록) 단위로 동작
- 상승 에지(Positive Edge)를 기준으로 신호 처리
- 모든 디자인은 동기화된 같은 클록 사용



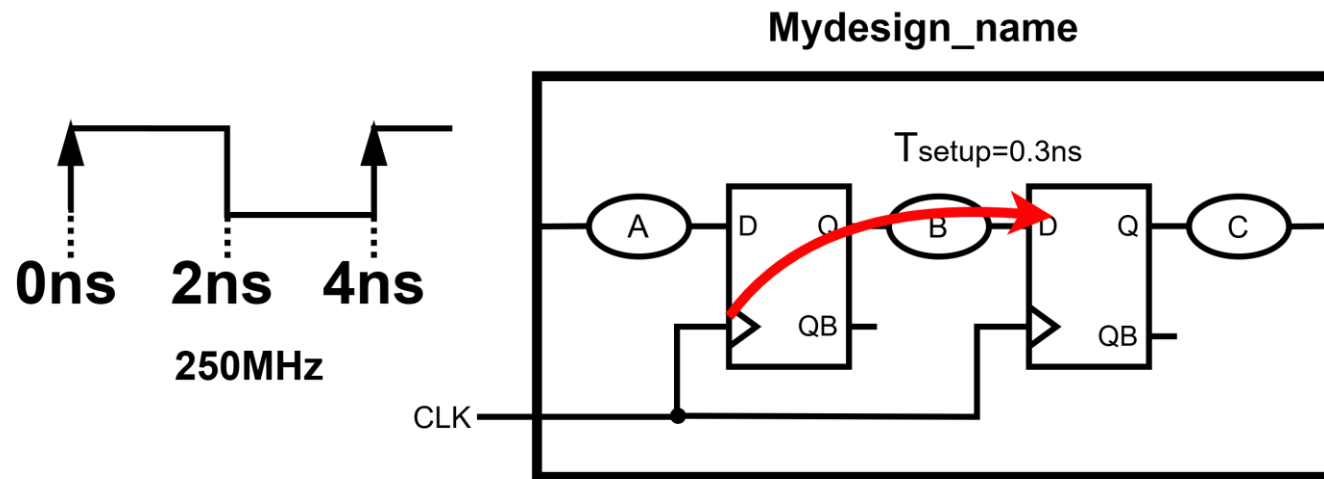
Internal Paths(Reg-to-Reg)



Internal Paths(Reg-to-Reg)

- Internal Paths(Reg-to-Reg)
 - 클럭 주기 설정

```
Genus> current_design Mydesign_name  
Genus> create_clock -period 4 [get_ports clk] # 4ns(250Mhz) 주기
```



클럭 주기 설정 = Internal Path(Reg-to-Reg)의 시간적 거리 부여

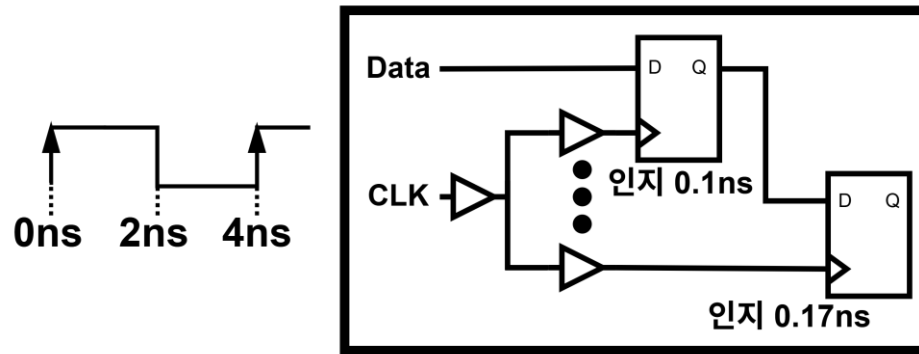
Internal Paths(Reg-to-Reg)

- Internal Paths(Reg-to-Reg)

- 클럭 Uncertainty(불확실성) : $Skew + Jitter + Margin$

- Skew: 각 Reg가 클럭을 인식하는 시점간의 차이(아래 그림)로, 세 요소 중 가장 높은 비중을 차지
- Jitter: 클럭 신호 위상의 미세한 변동. 이상적인 클럭 신호 대비 실제 신호가 시간축상으로 빠르거나 늦게 도착하여 좌우로 흔들려 보임
- Margin: 설계/제조 오차 대비 추가 여유 시간으로서, 쉽게 말해 **마진**

```
Genus> set_clock_uncertainty -setup 0.14 [get_clocks CLK]
Skew : 0.07ns
jitter : 0.01ns(+0.005ns, -0.005ns)
margin : 0.06ns
```



Skew : 플립플롭이 클럭을 인지하는 시점의 차이

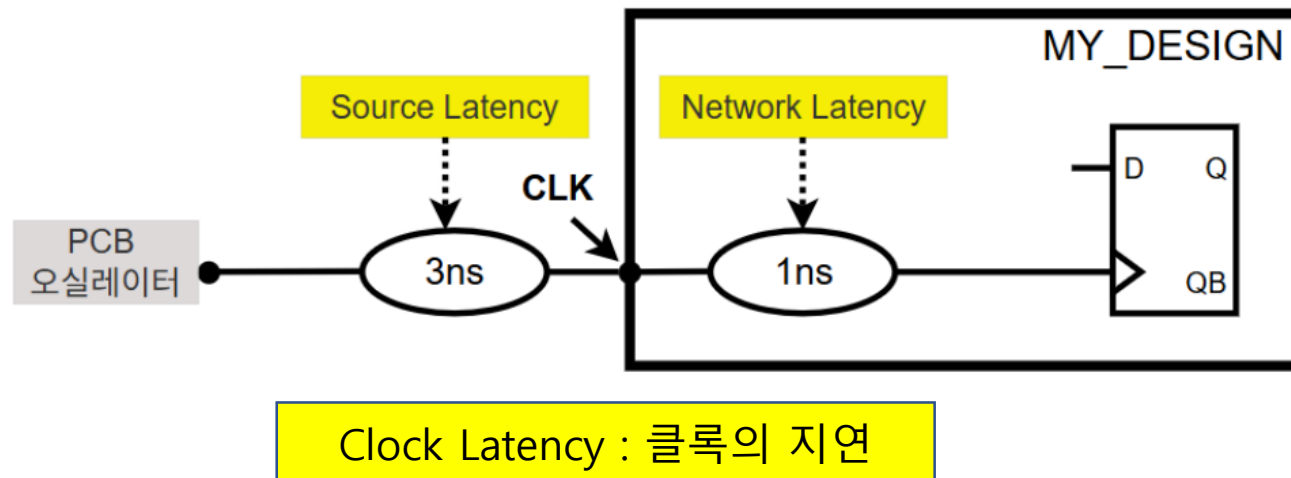
Internal Paths(Reg-to-Reg)

- Internal Paths(Reg-to-Reg)

- 클럭 Latency(지연)

- 클럭 신호가 전달되는 과정에서 발생하는 시간적 지연(delay)을 의미
 - 오실레이터(Oscillator)와 같은 클럭 소스에서 시작하여 내부의 레지스터까지 오는 동안 발생하는 모든 지연시간을 포함
 - **Source Latency** : PCB 상의 클럭 소스에서 칩까지의 지연
 - **Network Latency** : 칩 내부로서, 입력 포트부터 첫 번째 플립플롭까지의 지연

```
Genus> set_clock_latency -source -max 3 [get_clocks CLK]
Genus> set_clock_latency -max 1 [get_clocks CLK]
```

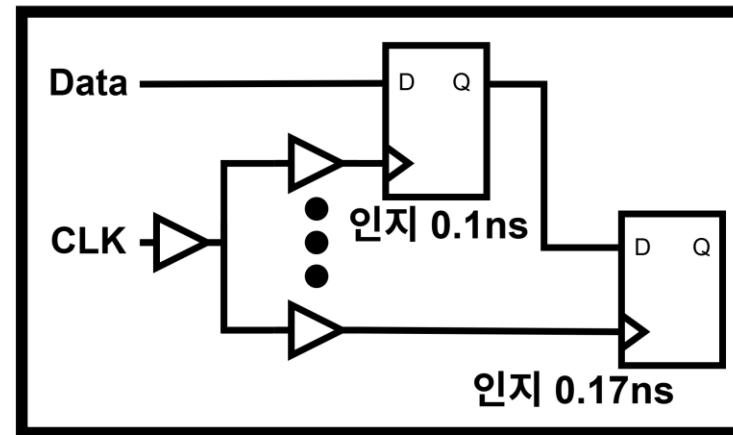
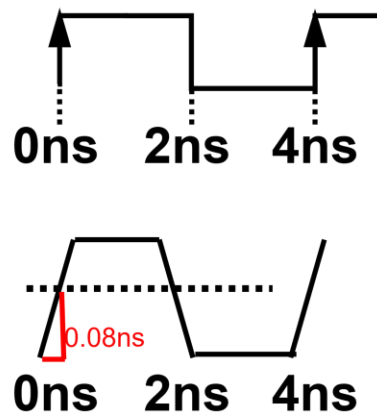


Internal Paths(Reg-to-Reg)

- Internal Paths(Reg-to-Reg)

- 클럭 Transition(기울기)
- 신호가 0에서 1로 상승하거나 1에서 0으로 하강할 때 걸리는 시간을 의미
- 쉽게 말하면 기울기, 어렵게 말하면 신호가 바뀌는 임계(Threshold)까지 걸리는 시간
- Transition이 길면 신호가 왜곡되거나 타이밍에 오류가 발생할 수 있으므로, 짧은 것이 상대적으로 유리

```
Genus> set_clock_transition 0.08 [get_clock CLK]
```

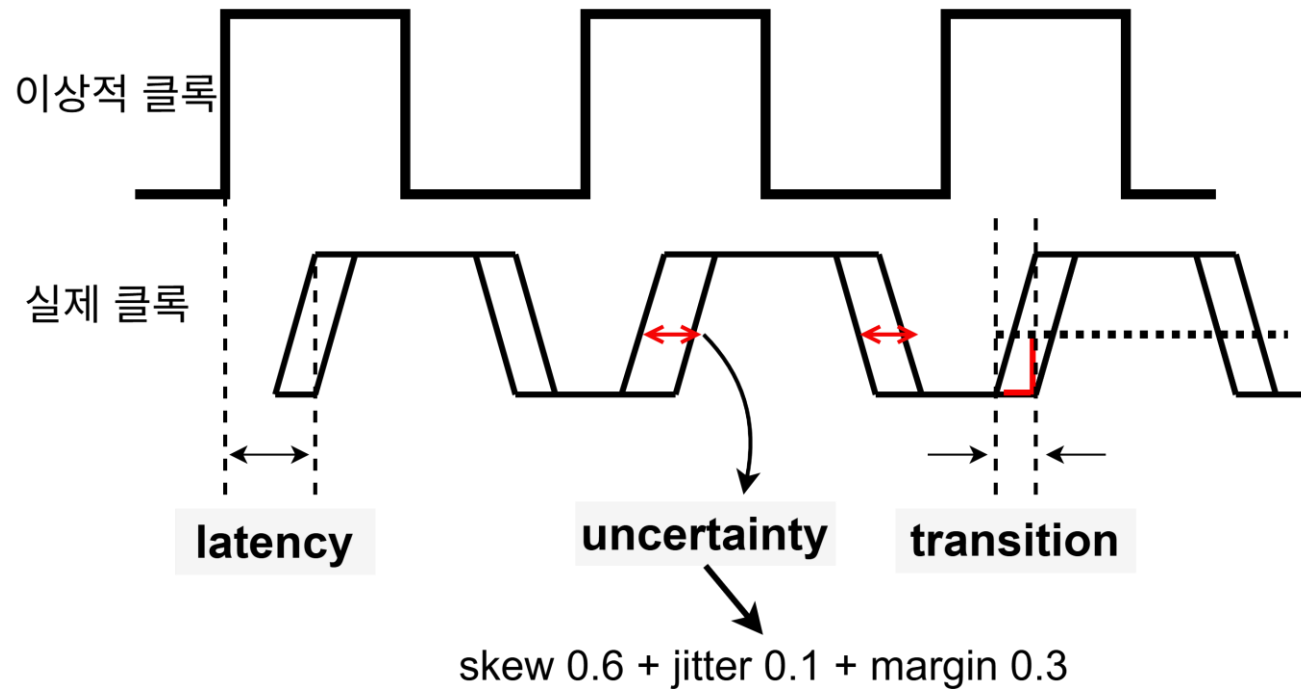


Transition : 임계점 도달까지의 소요시간(0.08ns)

Clock: Conclusion

- 클록에 대한 종합 정리

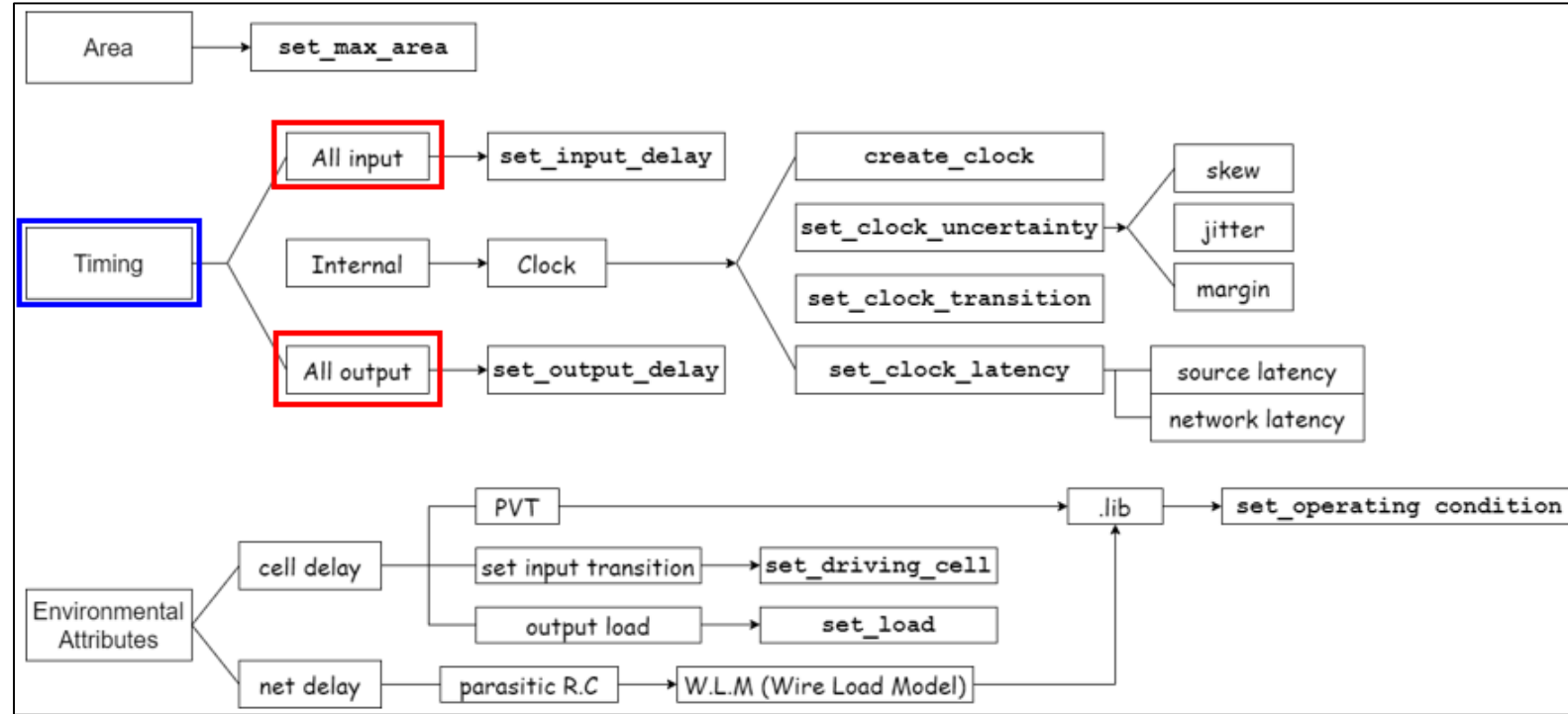
- 톨은 이 항목들을 설계자가 입력해주지 않으면 0으로 생각함
- Clock이 실제로 움직이는 수치를 부여할 수 있으니 신뢰성이 높은 디자인 제작가능



Input/Output Paths

- **Input/Output Paths**

- Input Path와 Output Path는 칩 입장에서 입, 출력에 해당한다.
- Constraint는 칩을 거시적으로 위에서 바라보았을 때 입, 출력 포트에 부여하는 것
- 즉, 내가 만들고 있는 디자인의 내부 회로나 Net에 관한 것이 아님

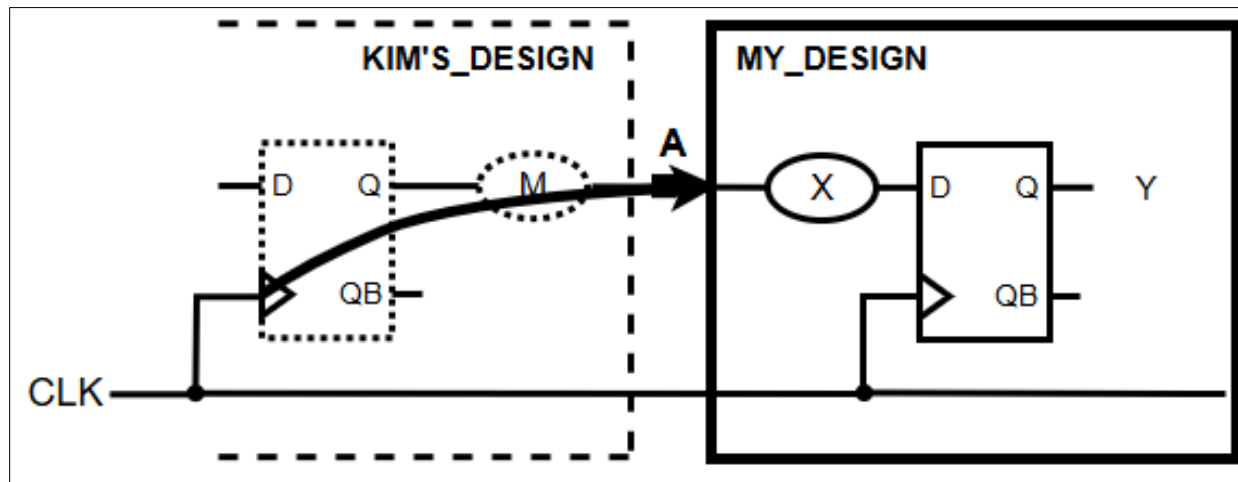


Input/Output Paths

- **Input Paths**

- 외부 신호가 칩 내부 레지스터로 전달되는 경로로서, 내 칩 바깥
- 아래 명령은 내 칩 바깥에서 0.3ns가 걸린다는 것을 나타냄

```
Genus> set_input_delay -max 0.3 -clock CLK [get_ports A]
```



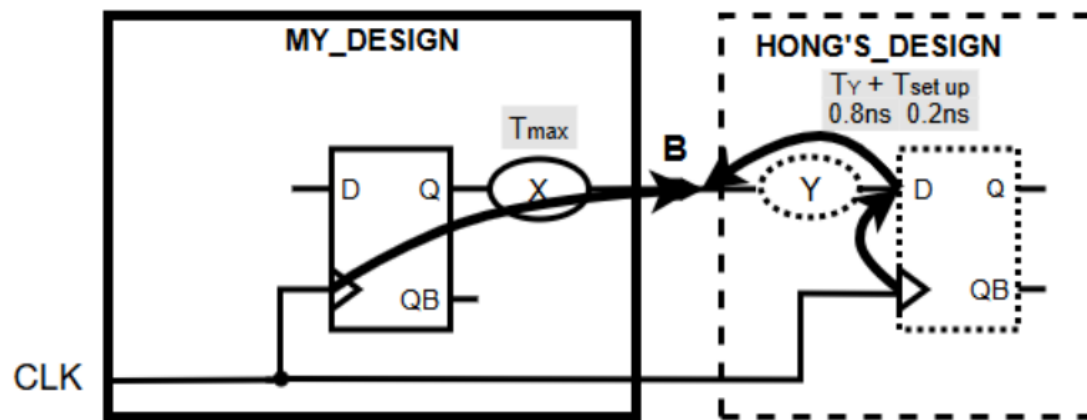
내 칩의 바깥. 칩 외부에서 소요되는 시간 설정

Input/Output Paths

• Output Paths

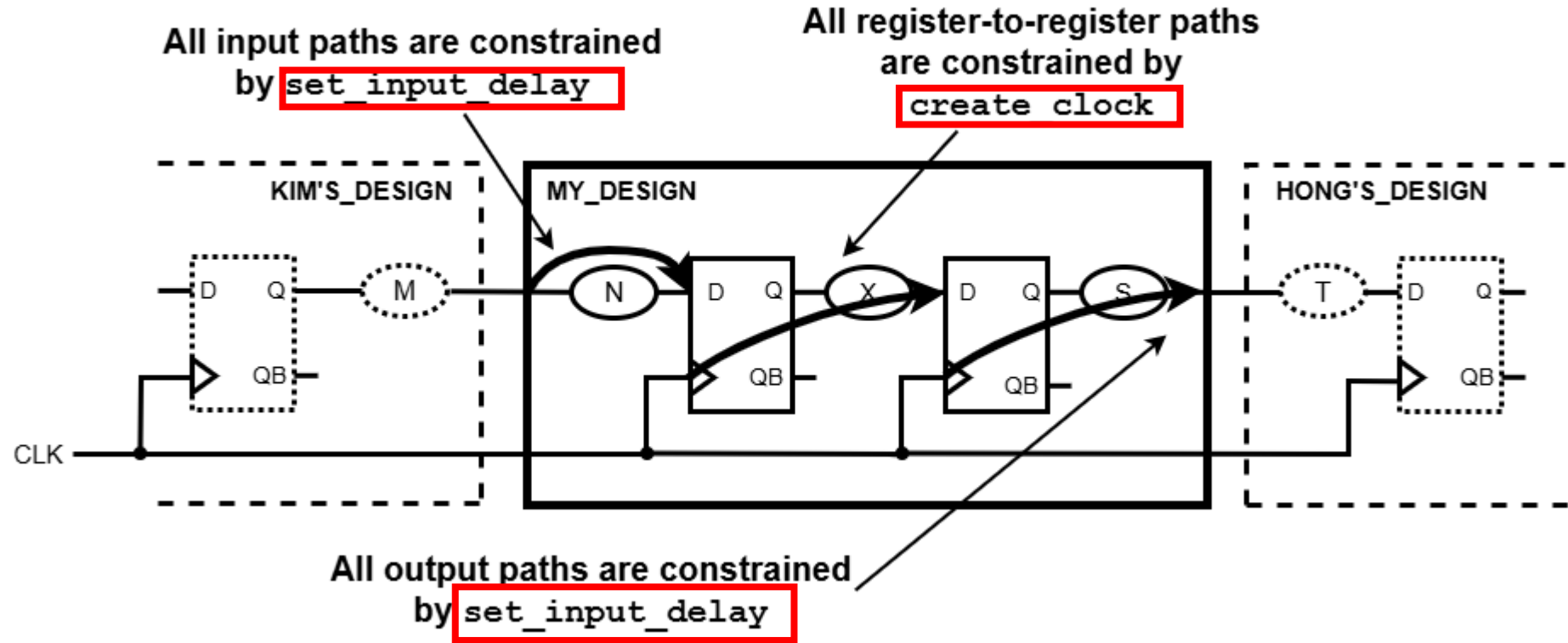
- 내부 레지스터에서 외부로 신호가 전달되는 경로로서, 내 칩 바깥
- 아래 예시 설명
 - Y라는 조합회로를 거치는 시간과 레지스터의 Setup 시간을 더하여 1ns를 부여함
 - 만약 클록 주기가 2ns라면, 칩 내부에서 $2 - 1 =$ 최대 1ns까지 신호가 머무르고 나가는 X 로직을 툴이 자동으로 생각하여 구현해 줌
 - 나는 아래처럼 명령 한 줄로 외부의 1ns를 말했을 뿐인데, 나머지를 툴이 알아서 계산해주는 것

```
Genus> set_output_delay -max 1 -clock CLK [get_ports B]
```



내 칩의 바깥. 칩 외부에서 소요되는 시간 설정

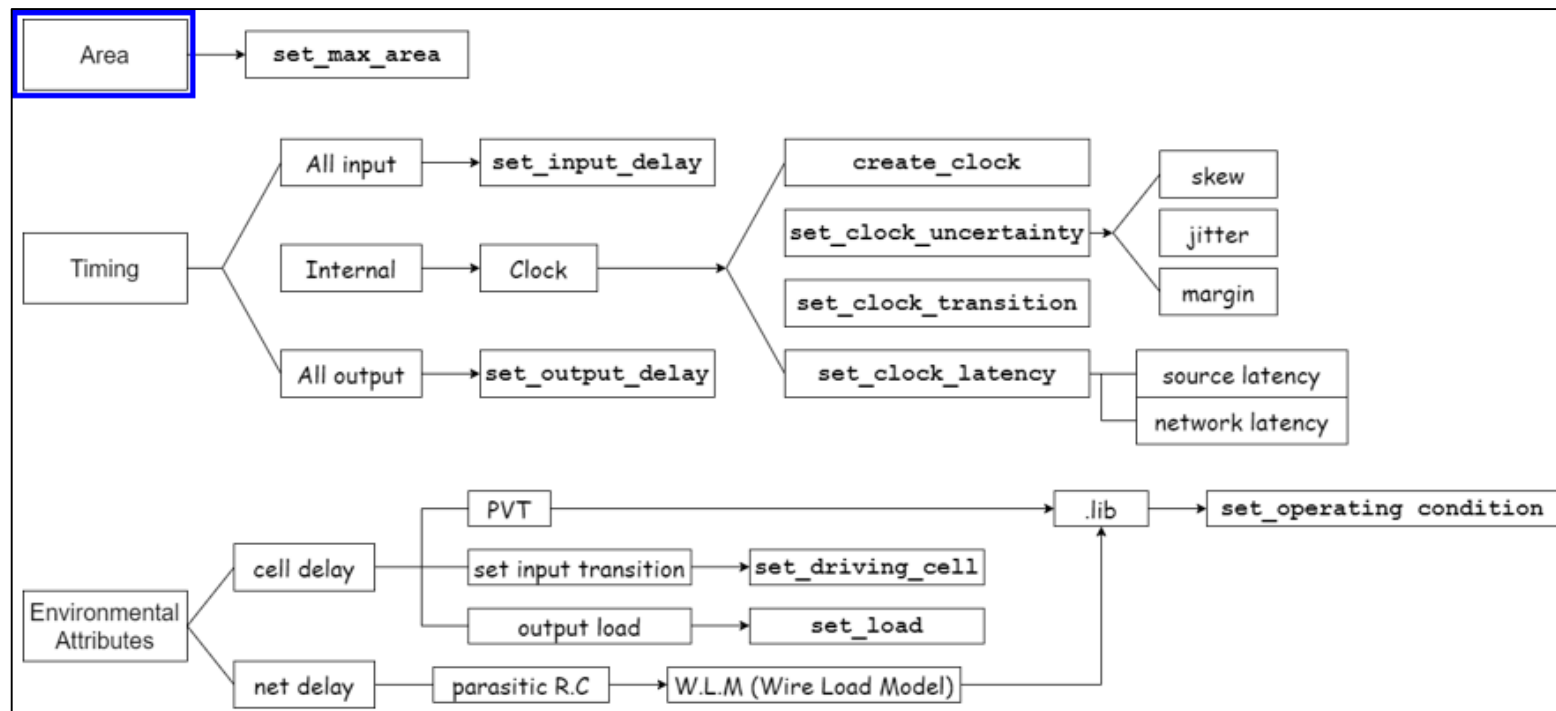
Timing Constraint: Conclusion



Area Constraint

- **Area Constraint**

- 설계된 회로가 특정 크기 내에서 구현되도록 제한하는 조건
- 면적이 제한된 환경에서 설계할 때 사용되며, 칩의 크기와 비용을 최적화하는 데 중요한 역할



Area Constraint

- **Area Constraint**

- 아래 숫자 1000은 가로와 세로의 곱인 μm^2 단위
- 2 input NAND 게이트의 크기를 문서를 통해 알아낸 뒤, 아래의 1000이라는 면적을 해당 크기로 나누어 몇 게이트인지 계산할 수 있음
- 예를 들어 2 input NAND 게이트 크기가 $10\mu\text{m}^2$ 라면, $1000/10 = 100$ 게이트 규모

```
Genus> set_max_area 1000 # 설계 면적을  $1000\mu\text{m}^2$ 로 제한
```

Environmental Attributes Constraint

- 칩이 물리적으로 만들어지고, PCB 보드에 실장된 뒤 어떤 환경에 놓이느냐를 생각해야 함
- 온도와 전압이 가장 큰 영향을 미침
- 사람도 여름과 겨울에 운동 능력이 다르고, 밥을 많이 먹은 경우와 적게 먹은 경우에 낼 수 있는 퍼포먼스가 다르다.
- 반도체도 온도와 전압에 따라 크게 영향을 받기 때문에 공정사 Foundry는 수많은 상황을 사전에 기본 소자들에 적용하고, 그로 인하여 얻어낸 결과를 라이브러리 .lib 파일로 제공하고 있다.
- 환경적인 요인 뿐만 아니라 자세한 공정 정보를 담고 있는 .lib 파일의 본질을 이해해야 함

Environmental Attributes Constraint

- 칩의 **Timing**에 영향을 미치는 요인들

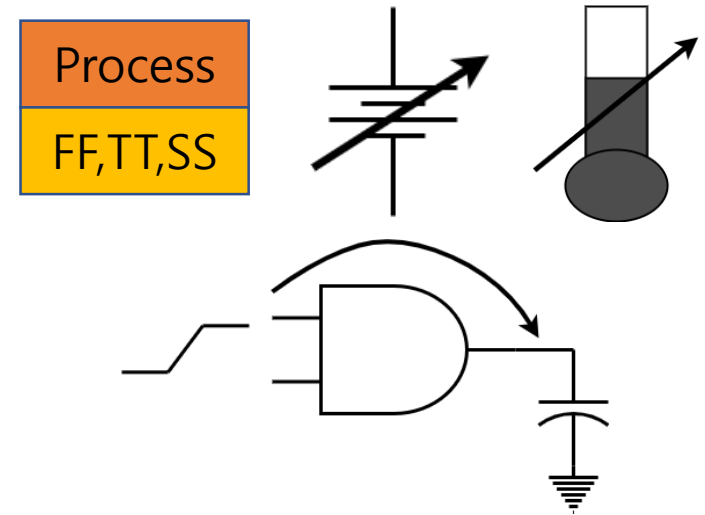
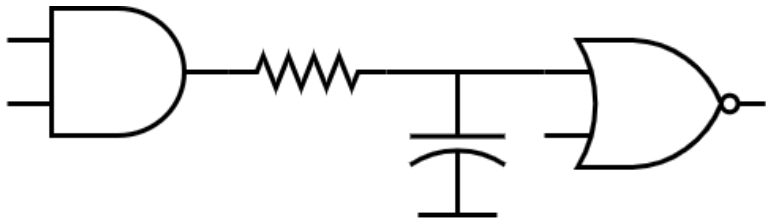
- Timing = Cell Delay + Net Delay

- Cell Delay**

- Process, Voltage, Temperature
 - 디자인의 입력에 대한 Input Transition(신호의 기울기)
 - 디자인의 출력에 대한 Output Load(Capacitance)

- Net Delay**

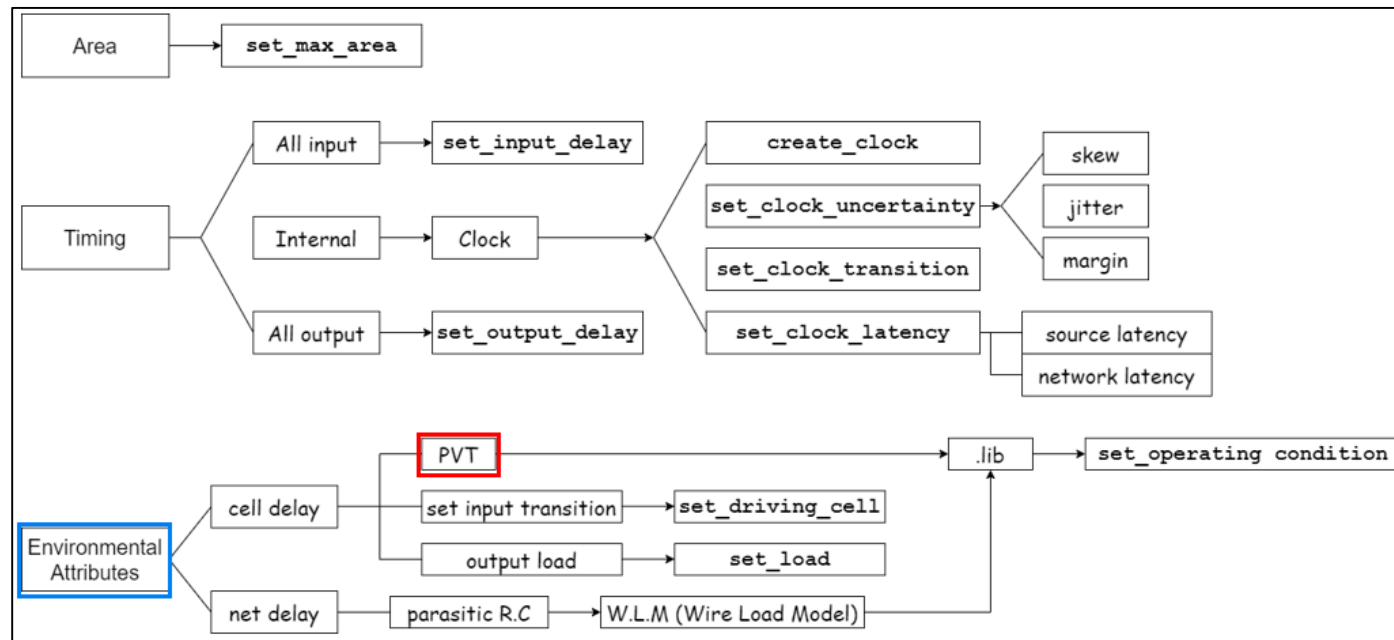
- Wire에 기생하고 있는 R, C
 - 디자인의 크기(Area)에 따라 다름



Cell Delay

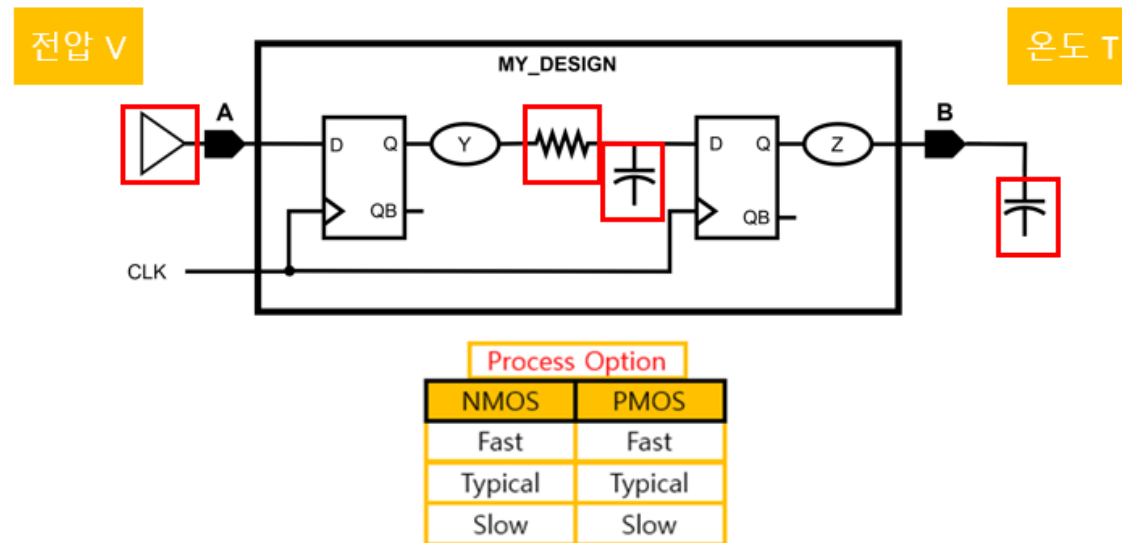
- **Cell Delay**

- 셀 딜레이에 영향을 미치는 요소
 - Process, Voltage, Temperature
 - 디자인의 입력에 대한 Input Transition
 - 디자인의 출력에 대한 Output Load



Cell Delay : PVT

- **PVT Corner(Process, Voltage, Temperature)**
 - 주황색은 디자인 밖의 요소로서 환경적인 요소
 - Process: 공정 변동 (Fast-Fast, Slow-Slow, Typical-Typical)
 - Voltage: 동작 전압 조건 (일반적으로 기준 전압의 $\pm 10\%$)
 - Temperature: 동작 온도 조건 ($-40^{\circ}\text{C} \sim 125^{\circ}\text{C}$)



Cell Delay : PVT

- 반도체의 기본 특성인 PVT Corner(Process, Voltage, Temperature)

- 온도

- 반도체 특성 : 저온에서 빠르게 동작, 반대로 고온에서 느리게 동작
- 같은 칩이더라도 남극과 북극지방에서는 빠른 동작이 가능하고, 화산 지역과 사막에서는 느리게 동작

- 전압

- 사람이 밥을 먹는 것과 비슷
- 배고프면 힘을 못 내서 느리게 동작, 반대로 밥을 많이 먹으면 힘을 많이 내서 퍼포먼스가 좋아짐
- 공정별로 정해져 있는 기준 전압을 문서에서 파악한 뒤 $\pm 10\%$ 를 적용함
- 예) 기준 전압이 1.0V라면, 0.9, 1.0, 1.1V가 가용 전압의 범위가 됨

- 공정

- 디지털 게이트는 주로 CMOS로 이루어짐(NMOS + PMOS)
- 셀을 빠르게 동작시킬 수 있는 공정을 FF(Fast-Fast), 느린 공정을 SS(Slow-Slow)라고 부름
- 디지털 설계에서는 SS, TT, FF 총 3가지 코너를 기본으로 고려한다

Cell Delay : PVT

- **PVT Corner(Process, Voltage, Temperature)**
 - 공정, 전압, 온도 정보는 .lib 파일에 모두 들어 있음
 - CADENCE GPDK045의 경우, 아래와 같은 .lib 파일들이 제공됨

```
fast_vdd1v0_basicCells.lib      report
fast_vdd1v0_extvdd1v0.lib      slow_vdd1v0_basicCells.lib
fast_vdd1v0_extvdd1v2.lib      slow_vdd1v0_extvdd1v0.lib
fast_vdd1v0_multibitsDFF.lib    slow_vdd1v0_extvdd1v2.lib
fast_vdd1v2_basicCells.lib      slow_vdd1v0_multibitsDFF.lib
fast_vdd1v2_extvdd1v0.lib      slow_vdd1v2_basicCells.lib
fast_vdd1v2_extvdd1v2.lib      slow_vdd1v2_extvdd1v0.lib
fast_vdd1v2_multibitsDFF.lib    slow_vdd1v2_extvdd1v2.lib
README.txt                     slow_vdd1v2_multibitsDFF.lib
```



```
=====
SUMMARY OF basicCells
##fast_vdd1v0_basicCells.lib : PVT_1P1V_0C
##fast_vdd1v2_basicCells.lib : PVT_1P32V_0C
##slow_vdd1v0_basicCells.lib : PVT_0P9V_125C
##slow_vdd1v2_basicCells.lib : PVT_1P08V_125C
=====
```


Cell Delay : PVT

- **PVT Corner(Process, Voltage, Temperature)**

- 설계자가 틀에서 해야 하는 일 (아래 순서대로)

- PVT 정보가 .lib에 들어 있음을 이해

- .lib 파일에 들어 있는 Operating Condition의 이름들과 특성들을 파악

- 합성 시 사용할 Operating Condition을 틀에게 알려주면 됨(아래 명령 사용)

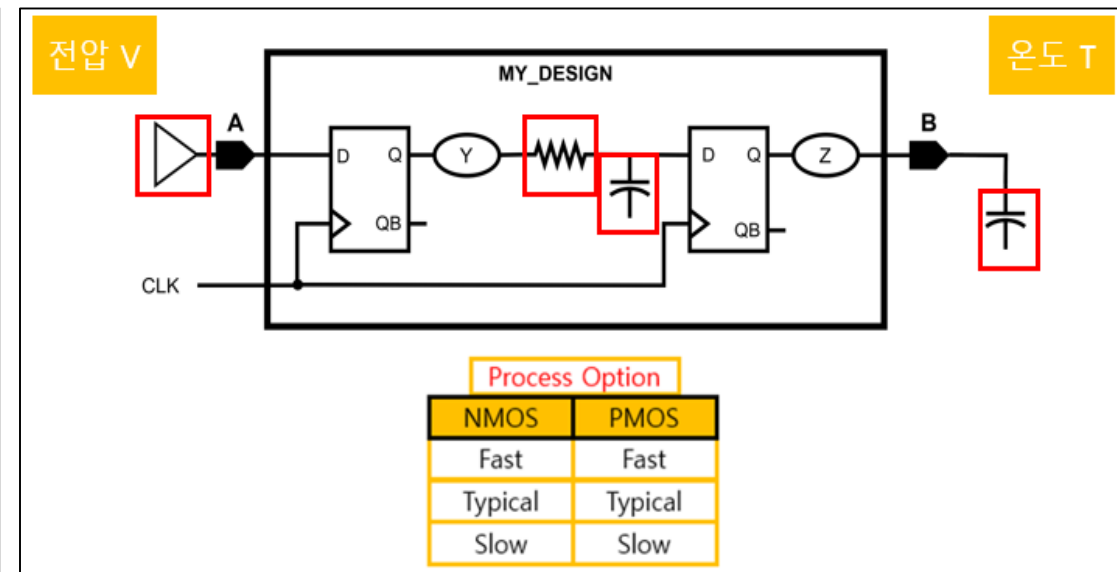
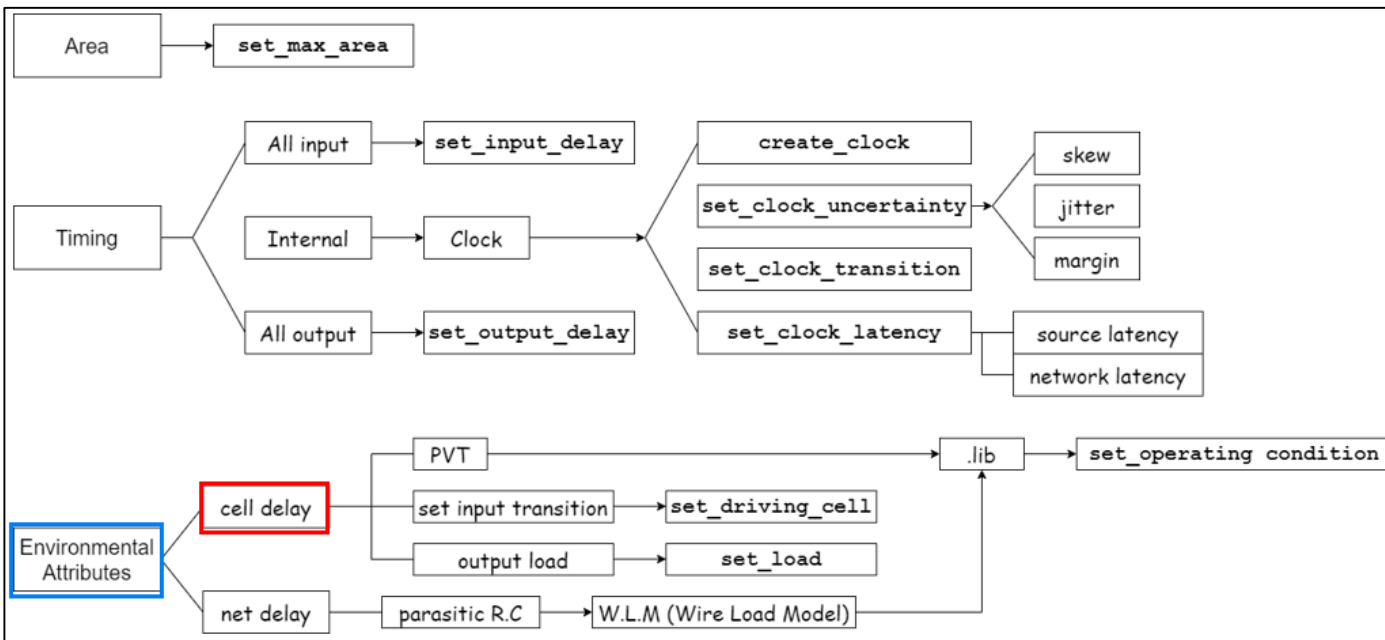


```
Genus> set_operating_condition PVT_0P9V_125C -library [get_libs $STD_LIB]
```

Cell Delay : Input transition / Output load

- **Cell Delay**

- 여러 PVT 중에 원하는 것을 설계자가 고르면 칩이 구동될 상황의 지정은 고정된 것
- 이제는 고정된 상황 내에서 Delay 즉, 얼마나 시간이 걸리는지 고려해야 함
- Cell Delay와 Net Delay의 합으로 생각할 수 있음

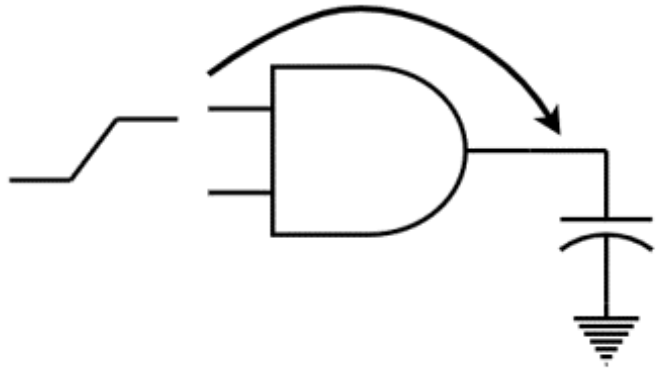


Cell Delay : Input transition / Output load

- **Input Transition+Output Load**

- 내 디자인을 AND 게이트라고 가정
- 입력으로 들어오는 신호는 실제로는 네모 반듯하지 않고, 기울기(Slew Rate)가 있다.
- 출력에는 눈에 보이지 않는 타이어가 있는 것처럼 Cap이 매달려 있다.
- 셀의 동작 시간은 이 두 가지에 영향을 받는다는 것이 중요

- 셀의 빠른 동작 : 입력으로 들어오는 신호의 기울기가 빠르고, 출력에 가벼운 타이어(Cap)가 있는 경우
- 셀의 느린 동작 : 입력으로 들어오는 신호의 기울기가 느리고, 출력에 무거운 타이어(Cap)가 있는 경우



Cell Delay에 영향을 주는 요소

Cell Delay : Input Transition

- **Input Transition**

- Input Transition 즉, 신호의 기울기 Slew Rate를 툴에게 알려주어야 함
 - 라이브러리 내의 특정 셀 중 하나가 내 디자인에 신호를 보내주고 있다고 말해주는 방법을 추천
 - 아래 명령은 내 디자인 RTL 코드에 PADDI 라는 입력 IO 셀을 실제로 사용하는 경우에 유용함

```
Genus> set_driving_cell PADDI -pin Y -library [get_libs $STD_LIB]
```

Cell Delay : Output Load

- **Output Load**

- 디자인의 출력에 있는 Cap을 뜻함. 운동선수로 치면 훈련용 타이어에 해당함
- 부하 (Load)가 가벼울수록 신호 출력 속도가 빨라지고, 클수록 신호 출력 속도가 느려짐
- 아래 명령은 내 디자인 RTL 코드에 PADDO 라는 출력 IO 셀을 실제로 사용하는 경우에 유용함

```
Genus> set_load [load_of PADDO/A] [all_outputs]
```

Cell Delay : Load Budget

- 수치 설정의 어려움

- Input Transition 과 Output Load 를 수치로 가늠할 수 없음
- 대안으로 셀을 사용하라고 했지만 어떤 셀을 선택해야 하는지 막막함

- 대안 : Load Budget

- 가늠하기 힘든 수치를 사용하지 말고, 셀을 사용할 것
 - 내 디자인에서 사용하는 IO 셀을 Driving 셀과 출력 Load(Cap)으로 선언
 - 장점 : 셀이라는 정확한 대상으로 느낄 수 있고 톨도 셀의 정보를 통해 수치를 자연적으로 인지할 수 있음

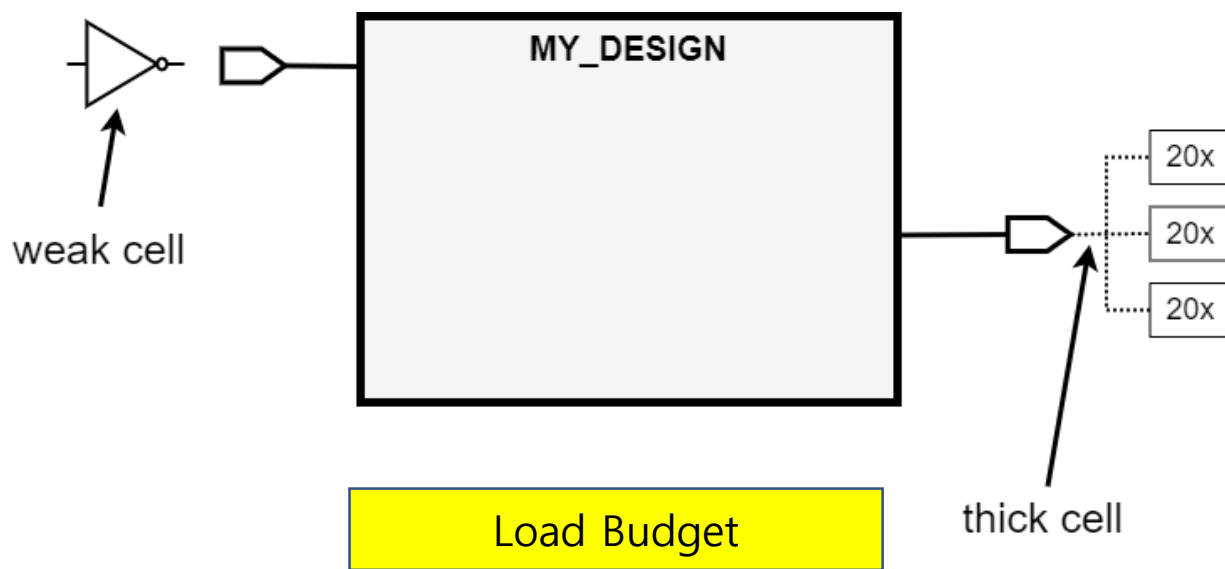
- Load Budget 사용

- Pessimistic 하게, 입력에 약한(weak) 셀이 신호를 보내고 있고, 출력에는 무거운 타이어가 달려 있다고 가정

Cell Delay : Load Budget

- **Load Budget의 실제 응용**

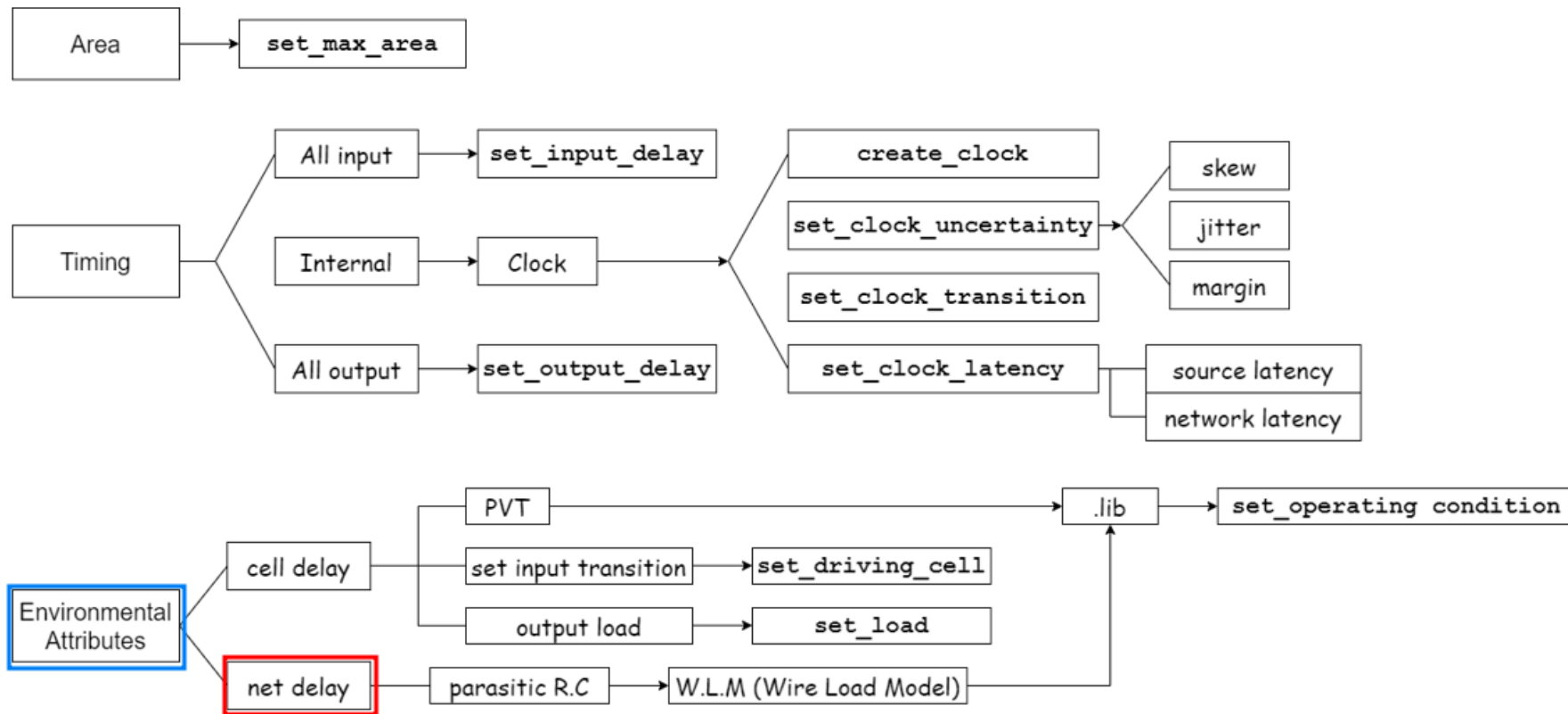
1. 수치보다는 공정사에서 제공한 셀 중을 선별하여 사용
2. Pessimistic 하게, 입력에 약한(weak) 셀이 신호를 보내고 있고, 출력에는 무거운 타이어가 달려 있다고 가정



Net Delay

- **Net Delay**

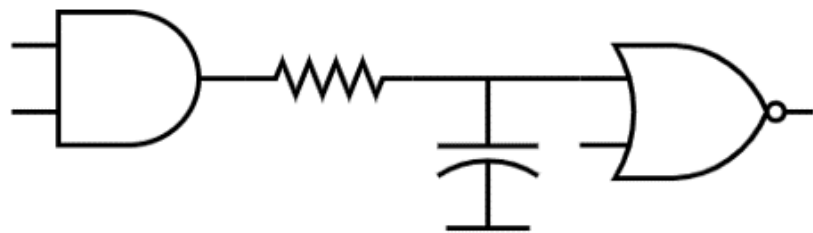
- 회로 내의 신호가 Net을 통해 전파되는 데 걸리는 시간



Net Delay

- **Net Delay**

- 배선에 존재하는 저항(Resistance)과 커패시턴스(Capacitance)에 의해 발생, 이를 기생 RC(Parasitic RC)라고 함
- 모든 Net에는 기생 RC 요소가 보이지 않지만 분명히 존재하며, RC 값이 클수록 신호 전달 속도가 느려지므로 이는 성능에 부정적인 영향을 미침
- RC 값은 배선 길이나 구조에 따라 달라질 수 있으며, 긴 경로에서는 더 큰 Net Delay가 발생한다.
- 아래 그림에서는 모든 Net에 기생하고 있는 RC를 나타내고 있다.



Net Delay 모델링

Net Delay

- **Net Delay**

- Q) 공정사에서는 Net Delay 정보를 어떻게 제공할까?
- A) 많은 시뮬레이션을 거친 뒤 결과를 통계치로 제공함. 통계치의 이름은 Wire Load Model(WLM)

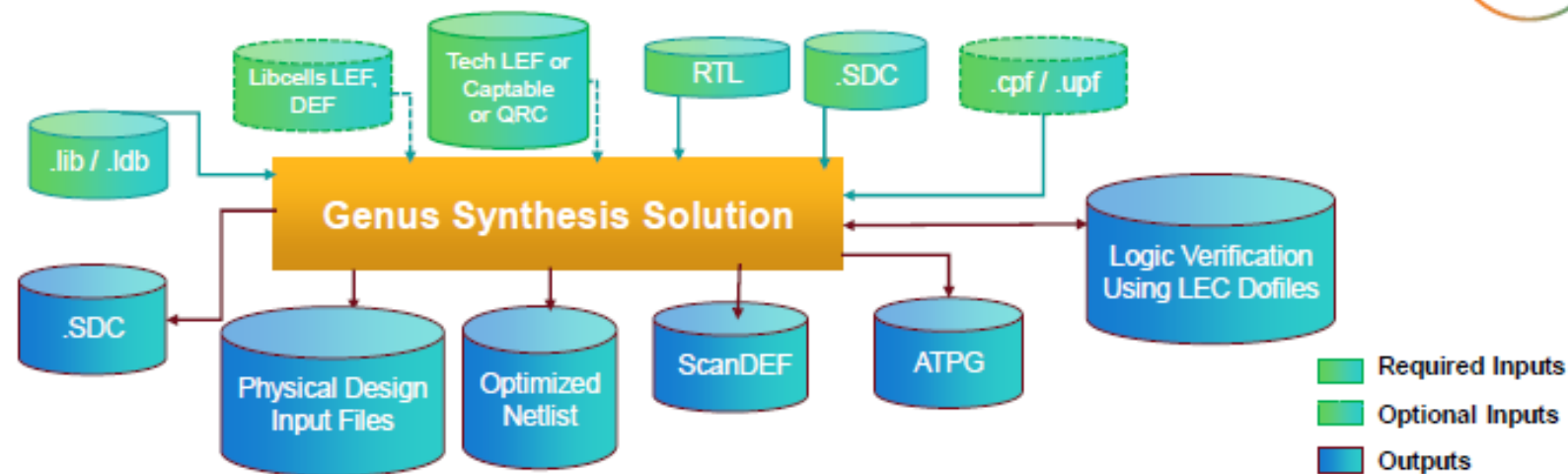
- **Wire Load Model**

- 공정사에서 제공하는 .lib 파일에 저장되어 있음
- 디자인의 크기에 따라서 Wire의 길이가 다르고, 결국 Wire에 걸리는 Load가 다르다는 것을 공정사가 잘 녹여내어 .lib 파일에 저장 및 제공함
- **툴이 우리 디자인의 크기(Area)를 가늠할 수 있음**. Area에 따라서 적합한 Wire Load Model을 선정하고 최종적으로 기생 R, C값과 최종 Net 딜레이까지 선정하여 타이밍 계산에 이용
- 아래 명령은 특별하게 설계자가 수동으로 지정하고 싶을 때 사용. 기본적으로는 자동 선정됨

```
Genus> set_wire_load_model -name <model_name> -library <library>
```

5. Preparation for Synthesis

Inputs and Outputs of Synthesis



Inputs	Outputs
RTL: Verilog, VHDL, directives, pragmas, SystemVerilog	Optimized netlist
Constraints: .sdc	LEC dofile
Library: .lib or .ldb	ATPG, ScanDEF, and others
Power Intent: CPF, UPF/IEEE 1801 (Optional)	Constraints: .sdc
Physical: Captable, QRC Technology file, LEF, DEF (Optional)	Physical design input files

Inputs for Synthesis: Standard Design Constraints(SDC)

- **SDC(Standard Design Constraints)**

- Clock Period, Pulse Width, Rise/Fall Times, in/output delays 를 기록
- Tcl(Tool Command Language)의 문법체계를 따름
- 설계가 실제 칩에서 동작을 할 수 있는지를 알기 위한 제약 사항 포함

```
create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.01 [get_ports "clk"]
set_input_delay -max 1.0 [get_ports "rst"] -clock [get_clocks "clk"]
set_output_delay -max 1.0 [get_ports "count"] -clock [get_clocks "clk"]
```

- create_clock -name -period 10 -waveform {0 5} {get_port "clk"}
 - Period: 10ns
 - Duty Cycle: 50%
 - Initial signal is high at first half
- set_clock_transition -rise / fall: transition delay
- set_clock_uncertainty: uncertainty for skew/jitter
- set_input / output delay: delay for timing slack calculation

Inputs for Synthesis: Liberty Timing File(.lib)

- **LIB(Liberty Timing File)**

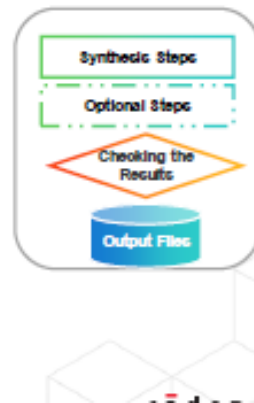
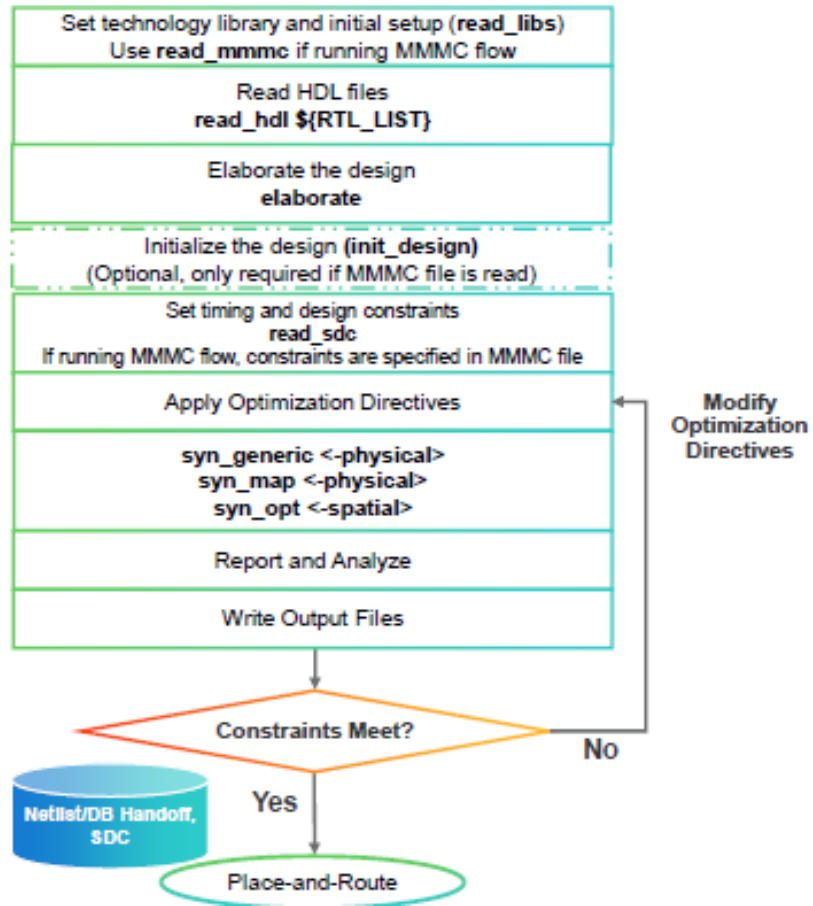
- ASCII 형태의 파일
- 특정 공정(process node)에서 Standard Cell의 timing과 power를 표현 함
- Standard cell library vendor나 공정사에서 제공함

- 참조: <https://teamvlsi.com/2020/05/lib-and-lef-file-in-asic-design.html>

```
cell (ADDFHX4) {
  area : 9.576;
  pg_pin (VDD) {
    pg_type : primary power;
    voltage name : "VDD";
  }
  pg_pin (VSS) {
    pg_type : primary ground;
    voltage name : "VSS";
  }
  leakage_power () {
    value : 291.623;
    when : "(A&B&CI)";
    related pg pin : VDD;
  }
  leakage_power () {
    value : 299.211;
    when : "(A&B&!CI)";
    related pg pin : VDD;
  }
  leakage_power () {
    value : 294.37;
    when : "(A&!B&CI)";
    related pg pin : VDD;
  }
  leakage_power () {
    value : 301.404;
    when : "(A&!B&!CI)";
    related pg pin : VDD;
  }
  leakage_power () {
    value : 301.142;
    when : "(!A&B&CI)";
    related pg pin : VDD;
  }
  leakage_power () {
    value : 304.68;
    when : "(!A&B&!CI)";
    related pg pin : VDD;
  }
  leakage_power () {
    value : 320.889;
    when : "(!A&!B&CI)";
  }
}

pin (C0) {
  direction : "output";
  function : "(((A B)+(B CI))+(CI A))";
  related_ground_pin : VSS;
  related_power_pin : VDD;
  max capacitance : 0.2;
  timing () {
    related_pin : "A";
    timing_sense : positive_unate;
    timing_type : combinational;
    cell_rise (delay_template 2x2) {
      index_1 ("0.008, 0.28");
      index_2 ("0.01, 0.2");
      values ( \
        "0.297343, 3.00601", \
        "0.437125, 3.14585" \
      );
    }
    rise_transition (delay_template 2x2) {
      index_1 ("0.008, 0.28");
      index_2 ("0.01, 0.2");
      values ( \
        "0.274732, 5.12281", \
        "0.275204, 5.12275" \
      );
    }
    cell_fall (delay_template 2x2) {
      index_1 ("0.008, 0.28");
      index_2 ("0.01, 0.2");
      values ( \
        "0.305941, 3.56188", \
        "0.43476, 3.69085" \
      );
    }
    fall_transition (delay_template 2x2) {
      index_1 ("0.008, 0.28");
      index_2 ("0.01, 0.2");
      values ( \
        "0.333299, 6.31161", \
        "0.33376, 6.31151" \
      );
    }
  }
}
```

Synthesis Flow



Synthesis Flow(1): Preset Global Variables and Attributes

```
#####  
## Preset global variables and attributes  
#####  
  
set DESIGN controller  
set GEN_EFF medium  
set MAP_OPT_EFF high  
set DATE [clock format [clock seconds] -format "%b%d-%T"]  
set _OUTPUTS_PATH outputs_${DATE}  
set _REPORTS_PATH reports_${DATE}  
set _LOG_PATH logs_${DATE}  
exec mkdir -p $_LOG_PATH  
##set MODUS_WORKDIR <MODUS work directory>  
set_db / .init_lib_search_path {.  
set_db / .script_search_path {.  
set_db / .init_hdl_search_path {.  
##Uncomment and specify machine names to enable super-threading.  
##set_db / .super_thread_servers {<machine names>}  
##For design size of 1.5M - 5M gates, use 8 to 16 CPUs. For designs > 5M gates, use 16 to 32 CPUs  
##set_db / .max_cpus_per_server 8  
  
##Default undriven/unconnected setting is 'none'.  
##set_db / .hdl_unconnected_value 0 | 1 | x | none  
  
set_db / .information_level 7
```

- Cadence RTL-to-GDSII에 관여하는 모든 툴들은 1) 일관된 Stylus UI와 2) 일관된 TCL 명령어 체계를 가짐
- 디자인 프로젝트 이름의 설정
- 스크립트 파일의 재활용을 위한 각종 글로벌 변수의 설정

Synthesis Flow(2): Library Setup

```
#####  
## Library setup  
#####  
  
read_libs muddlib.lib  
read_physical -lef muddlib.lef  
## Provide either cap_table_file or the qrc_tech_file  
#set_db / .cap_table_file <file>  
#read_qrc <qrcTechFile name>  
  
#set_db / .lp_insert_clock_gating false
```

- Liberty file(.lib) import
- Library Exchange Format(.lef) import

Synthesis Flow(3): Constraints Setup

```
#####  
## Constraints Setup  
#####  
  
read_sdc constraints.sdc  
puts "The number of exceptions is [llength [vfind "design:$DESIGN" -exception *]]"  
  
if {[file exists ${_OUTPUTS_PATH}]} {  
    file mkdir ${_OUTPUTS_PATH}  
    puts "Creating directory ${_OUTPUTS_PATH}"  
}  
  
if {[file exists ${_REPORTS_PATH}]} {  
    file mkdir ${_REPORTS_PATH}  
    puts "Creating directory ${_REPORTS_PATH}"  
}
```

- Constraints File(.sdc) Import

Synthesis Flow(4): Synthesizing to Generic(Translation)

```
#####  
## Synthesizing to generic  
#####  
  
set_db / .syn_generic_effort $GEN_EFF  
syn_generic  
puts "Runtime & Memory after 'syn_generic'"  
time_info GENERIC  
report_dp > $_REPORTS_PATH/generic/${DESIGN}_datapath.rpt  
write_snapshot -outdir $_REPORTS_PATH -tag generic  
report_summary -directory $_REPORTS_PATH
```

- RT Level의 HDL 코드를 Generic Gate Level HDL로 변환
- Simple Translation

Synthesis Flow(5): Synthesizing to Gate(Mapping)

```
#####  
## Synthesizing to gates  
#####  
  
set_db / .syn_map_effort $MAP_OPT_EFF  
syn_map  
puts "Runtime & Memory after 'syn_map'"  
time_info MAPPED  
write_snapshot -outdir $REPORTS_PATH -tag map  
report_summary -directory $REPORTS_PATH  
report_dp > $_REPORTS_PATH/map/${DESIGN}_datapath.rpt
```

- Generic Gate를 실제 Standard Cell Library상의 Gate와 mapping

Synthesis Flow(6): Optimize Netlist(Optimization)

```
#####  
## Optimize Netlist  
#####  
  
## Uncomment to remove assigns & insert tiehilo cells during Incremental synthesis  
##set_db / .remove_assigns true  
##set_remove_assign_options -buffer_or_inverter <libcell> -design <design|subdesign>  
##set_db / .use_tiehilo_for_const <none|duplicate|unique>  
set_db / .syn_opt_effort $MAP_OPT_EFF  
syn_opt  
write_snapshot -outdir $_REPORTS_PATH -tag syn_opt  
report_summary -directory $_REPORTS_PATH  
  
puts "Runtime & Memory after 'syn_opt'"  
time_info OPT  
  
write_snapshot -outdir $_REPORTS_PATH -tag final  
report_summary -directory $_REPORTS_PATH  
write_hdl > ${_OUTPUTS_PATH}/${DESIGN}_m.v  
write_design -innovus  
## write_script > ${_OUTPUTS_PATH}/${DESIGN}_m.script  
write_sdc > ${_OUTPUTS_PATH}/${DESIGN}_m.sdc
```

- PPA(Power Performance Area)를 고려하여 최적화된 Gate Level Netlist를 도출

Synthesis Flow(7): Write "*do lec*" file

```
#####  
## write do lec  
#####  
  
write_do_lec -golden_design fv_map -revised_design ${_OUTPUTS_PATH}/${DESIGN}_m.v -logfile ${_LOG_PATH}/intermediate2  
final.lec.log > ${_OUTPUTS_PATH}/intermediate2final.lec.do  
##Uncomment if the RTL is to be compared with the final netlist..  
##write_do_lec -revised_design ${_OUTPUTS_PATH}/${DESIGN}_m.v -logfile ${_LOG_PATH}/rtl2final.lec.log > ${_OUTPUTS_PATH}/rtl2final.lec.do
```

- Equivalence Checking을 위한 do_lec파일 생성
- Equivalence Checking: RTL-to-GDII Flow 과정간 Golden Netlist와 Revised Netlist 간의 논리적 등가성(logical equivalence) 확인

Start Genus(1)



```
genus [-abort_on_error] [-batch]
      [-del_scale 10]
      [-disable_user_startup] [-
execute <string>]+ [-files
<string>]+
      [-help] [-legacy_ui] [-lic_stack
<integer>] [-lic_startup
<string>]
      [-lic_startup_options <string>]+
      [-log <prefix>] [-no_gui]
      [-legacy_gui] [-overwrite] [-
version] [-wait <integer>]
```

- Genus Synthesis Solution은 세 가지 모드로 운영가능
 1. genus 구동 후 interactive mode로 command 입력
 2. tcl script를 이용하여 genus 구동
 3. genus 구동 후 tcl script 불러오기 (sourcing)

Start Genus(2)

```
william@npit synthesis]$ genus
TMPDIR is being set to /tmp/genus temp_5879_npit.ic.rnd1_william_Xpr5ku
Cadence Genus(TM) Synthesis Solution.
Copyright 2024 Cadence Design Systems, Inc. All rights reserved worldwide.
Cadence and the Cadence logo are registered trademarks and Genus is a trademark
of Cadence Design Systems, Inc. in the United States and other countries.

[17:13:43.440217] Configured Lic search path (22.01-s003): 35266@npit-service1.iptime.org

Version: 22.16-s078_1, built Sun Jun 09 22:32:57 PDT 2024
Options:
Date: Wed Aug 14 17:13:43 2024
Host: npit.ic.rnd1 (x86_64 w/Linux 3.10.0-1160.el7.x86_64) (20cores*80cpus*2physical cpus*Intel(R)
(R) Silver 4316 CPU @ 2.30GHz 30720KB) (395263124KB)
PID: 5879
OS: Red Hat Enterprise Linux Server release 7.9 (Maipo)

Checking out license: Genus_Synthesis
[17:13:43.082563] Periodic Lic check successful
[17:13:43.082571] Feature usage summary:
[17:13:43.082576] Genus_Synthesis

*****
*
*****
*

Finished executable startup (0 seconds elapsed).

Loading tool scripts...
Finished loading tool scripts (12 seconds elapsed).

@genus:root: 1> set db / .init
init_blackbox_for_undefined init_delete_floating_hnets init_design_mmmc_skip_inactive
init_ground_nets init_hdl_search_path init_keep_empty_modules
init_lef_files init_lib_phys_consistency_checks init_lib_search_path
init_min_dbu_per_micron init_mmmc_version init_oa_abstract_views
init_oa_default_rule init_oa_layout_views init_oa_ref_libs
init_oa_search_libs init_oa_special_rule init_physical_only
init_power_intent_files init_power_nets init_prototype_design
init_state init_sync_relative_path init_timing_enabled
@genus:root: 1> set db / .init_hdl_search_path {.}
```

```
genus.cmd          genus.log          genus_dft_script.tcl*
genus.cmd1         genus.log1         genus_script.tcl*
[william@npit synthesis]$ genus -f genus_script.tcl -log synth1.log
```

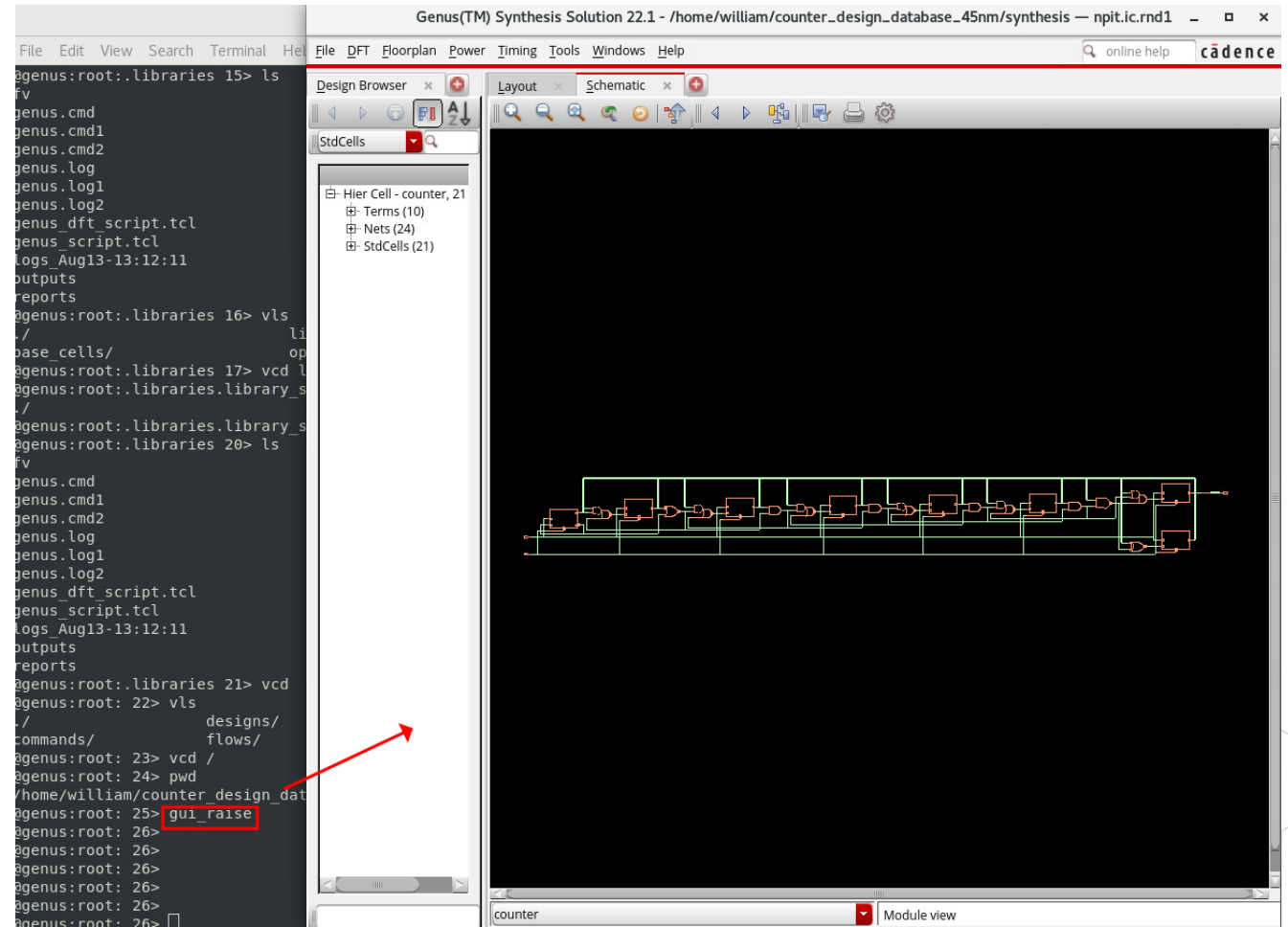
```
genus.cmd          genus.cmd1 genus.cmd2 genus.log genus.log1 genus.log2 genus_dft_script.tcl
genus_script.tcl
@genus:root: 1> source genus_script.tcl
```


Raise GUI



gui_show

The main GUI window has the follow



Setting Attribute[s]



```
set_db <object>  
.<attribute_name> <value>
```

```
set_db / .<attr_name> <value>  
      =  
set_db <attr_name> <value>
```

- Genus Program에 미리 정의된 변수에 값을 할당
- Read and Write 권한을 부여
 - 일부 속성은 read-only 권한

Querying Attribute[s]



```
get_db <object> .<attribute_name>
```

- set_db 로 설정된 속성(attribute)을 조회
- 단일속성 혹은 복수의 속성을 한번에 조회 가능

[Example]

```
get_db tns_opto
```

```
get_db "clock:map_design/functional/high_clk clock:map_design/functional/low_clk" .source
```

Reading Libraries



```
read_libs [-min_libs string] [-max_libs string] [-aocv_libs string]
[-socv_libs string] [filenames...]
```

- init_lib_search_path에서 정의한 라이브러리 경로에서 라이브러리 파일(.lib 또는 .v)을 읽어 들인다

```
@genus:root: 1> set_db init_lib_search_path ../lib
Setting attribute of root '/': 'init_lib_search_path' = ../lib
1 ../lib
@genus:root: 2> read_libs slow_vdd1v0_basicCells.lib
```

Reading Designs(RTL)



`read_hdl/read_netlist`

- `init_hdl_search_path`에서 정의한 HDL파일 경로에서 RTL Source(Verilog, System Verilog, VHDL, SystemC 등)를 읽어 들인다.
- 복수의 파일일 경우 {design1 design2 }와 같은 방식으로 { } 안에 스페이스를 delimiter로 해서 enumeration한다.
- verilog source file의 경우 default는 v1995, v2001이다

```
@genus:root: 3> set_db init_hdl_search_path ../rtl
Setting attribute of root '/': 'init_hdl_search_path' = ../rtl
1 ../rtl
@genus:root: 4> read_hdl i2c.v
```

Elaboration



```
elaborate [-h] [-parameters {} ] [<top_module_name>]
```

- 데이터 구조를 형성
- 디자인 상에 필요한 register를 추론
- 합성되지 않는 코드의 제거를 통한 최적화
- Gate Clock의 대상 블록을 확인

```
@genus:root: 4>  
@genus:root: 4> elaborate $DESIGN
```

Read Constraints



```
read_sdc [-stop_on_errors] [-view <analysis_view>] \  
[-no_compress] <sdcFileName>
```

- Elaboration 이후에 수행
- Standard Design Constraints file(.sdc)를 읽어 들임

```
constraints/  copyright  
@genus:root: 4> read_sdc ../constraints/constraints_top.sdc
```

Setting Effort Levels Prior to Synthesis

```
set_db syn_generic_effort medium  
set_db syn_map_effort medium  
set_db syn_opt_effort medium
```

- Synthesis 3-Stage에 대한 Effort Level을 설정한다
- 변수는 preset 단계에서 global하게 설정할 수 있다
 - 필요한 경우 override하여 적용한다

Generic Optimization

`syn_generic <-
physical>`

Technology
Transformation
(Mapping)

`syn_map <-
physical>`

Incremental
Optimization

`syn_opt <-
spatial> <-incr>`

Generating Reports

Command	Description
<code>report_area</code>	Prints an exhaustive hierarchical area report.
<code>report_dp</code>	Prints a datapath resources report (to be done before <code>syn_map</code>).
<code>report_design_rules</code>	Prints design rule violations.
<code>report_gates</code>	Reports libcells used, total area, and instance count summary.
<code>report_hierarchy</code>	Prints a hierarchy report.
<code>report_instance</code>	Generates a report on the specified instance.
<code>report_memory</code>	Prints a memory usage report.
<code>report_messages</code>	Prints a summary of the error messages that have been issued.
<code>report_power</code>	Prints a power report.
<code>report_qor</code>	Prints a quality-of-results report.
<code>report_timing</code>	Prints a timing report.
<code>report_summary</code>	Prints an area, timing, and design rules report.

```
#reports
report_timing > reports/report_timing.rpt
report_power  > reports/report_power.rpt
report_area   > reports/report_area.rpt
report_qor    > reports/report_qor.rpt
```



```
[william@npit reports]$ ls -al
total 20
drwxr-xr-x.  2 william admin  120 Aug 14 17:49 .
drwxr-xr-x. 10 william admin 4096 Aug 14 18:15 ..
-rw-r--r--.  1 william admin 2705 Aug 14 17:49 report_area.rpt
-rw-r--r--.  1 william admin 1204 Aug 14 17:49 report_power.rpt
-rw-r--r--.  1 william admin 1744 Aug 14 17:49 report_qor.rpt
-rw-r--r--.  1 william admin 2950 Aug 14 17:49 report_timing.rpt
```

Generating Outputs

Command	Details
<code>write_db</code>	This command writes the design into a database file which is used to reload the design into the genus tool.
<code>write_netlist</code>	This command generates a gate-level-netlist file which is used in the Innovus tool.
<code>write_script</code>	This command generates a Genus constraints file which is used to reload into Genus and optimize the results.
<code>Write_sdc</code>	This command generates an sdc constraints file which is used in the Innovus tool.
<code>Write_design</code>	This command generates all the files needed to reload the session in Genus and Innovus.

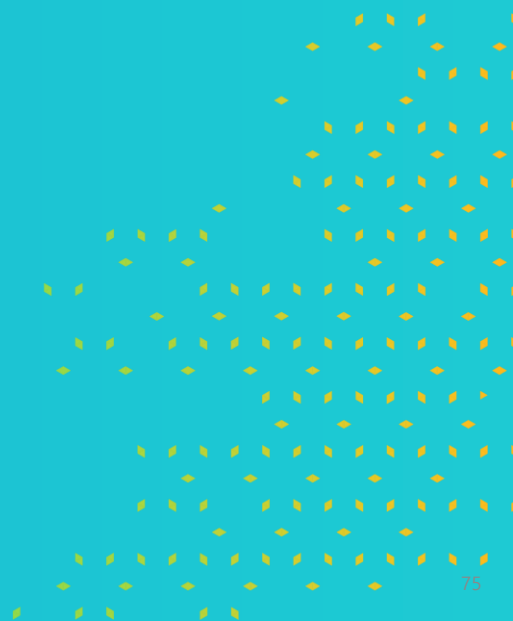
```
#Outputs
write_hdl > outputs/process_multi_netlist.v
write_sdc > outputs/processor_multi_sdc.sdc
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -setuphold split > outputs/delays.sdf
```



```
drwxr-xr-x.  2 william admin   102 Aug 13 12:46 .
drwxr-xr-x. 10 william admin  4096 Aug 14 18:25 ..
-rw-r--r--.  1 william admin 41742 Aug 14 17:49 delays.sdf
-rw-r--r--.  1 william admin 15577 Aug 14 17:49 process_multi_netlist.v
-rw-r--r--.  1 william admin  5542 Aug 14 17:49 processor_multi_sdc.sdc
```



LAB



LAB(1)

- Interactive Mode Synthesis

LAB(2)

- TCL Scripting for Synthesis